

Beyond Synthetic Benchmarks: Evaluating LLM Performance on Real-World Class-Level Python Code Generation

Musfiquir Rahman · SayedHassan
Khatoonabadi · Emad Shihab

Received: date / Accepted: date

Abstract Large language models (LLMs) have advanced rapidly from well-studied function-level code generation to emerging repository-level approaches, yet this progression reveals a critical gap: class-level code generation, which is the fundamental building block connecting individual functions to complete repositories, remains systematically underexplored. Understanding class-level capabilities is a prerequisite to repository-level success, as current approaches cannot distinguish whether failures stem from basic implementation deficiencies or sophisticated context management issues.

We introduce *RealClassEval*, a benchmark derived from real-world open-source repositories, comprising 400 classes divided into seen and unseen partitions based on model training cutoffs. Our systematic evaluation across seven state-of-the-art LLMs reveals fundamental limitations masked by synthetic benchmarks: while LLMs achieve 84–89% correctness on synthetic tasks, they attain only 25–34% on real-world classes, with minimal distinction between seen and unseen codebases.

Notably, this performance range converges with concurrent work using rich natural language descriptions and repository context, suggesting that specification complexity cannot overcome core implementation bottlenecks. Documentation provides marginal improvements (1–3%), while retrieval augmentation yields greater gains (4–7%,) specifically with incomplete specifications. Error analysis reveals `AttributeError`, `TypeError`, and `AssertionError` domi-

Musfiquir Rahman
Concordia University, Montréal, QC, Canada
E-mail: musfiquir.rahman@mail.concordia.ca

SayedHassan Khatoonabadi
Concordia University, Montréal, QC, Canada
E-mail: sayedhassan.khatoonabadi@concordia.ca

Emad Shihab
Concordia University, Montréal, QC, Canada
E-mail: emad.shihab@concordia.ca

nate 84% of failures, with distinct patterns between synthetic (logic-focused) and real-world (object-oriented) scenarios.

These findings establish the diagnostic foundation necessary for advancing both fundamental LLM capabilities and sophisticated repository-level approaches, while providing actionable guidance for production deployment strategies.

Keywords Large Language Models · Code Generation · Software Repository Mining · Benchmark Dataset

1 Introduction

Large language models (LLMs) have demonstrated remarkable progress in code generation, enabling tools like GitHub Copilot (git, 2023) and Amazon CodeWhisperer (ama, 2024) to assist developers across multiple granularities (Chen et al., 2021; Tao et al., 2025). The field has evolved rapidly from well-studied function-level generation (Chen et al., 2021; Austin et al., 2021) to emerging repository-level approaches (Jimenez et al., 2023; Zheng et al., 2024; Zhang et al., 2024) that tackle complex multi-file scenarios. However, this progression reveals a critical gap: **class-level code generation**, which is the fundamental building block connecting individual functions to complete repositories, remains systematically underexplored.

This gap has profound implications for both understanding LLM capabilities and advancing repository-level approaches. Current repository-level benchmarks like SWE-bench (Jimenez et al., 2023) and recent advances in environment-aware generation (Wu et al., 2026) implicitly assume successful class-level implementation capabilities. Yet when repository-level approaches fail, we cannot determine whether failures stem from class-level implementation deficiencies, context management issues, or dependency resolution challenges. Without understanding baseline class-level capabilities, researchers cannot effectively focus their architectural innovations or set realistic expectations for repository-level performance.

Existing class-level evaluation relies primarily on **ClassEval** (Du et al., 2023), which uses manually crafted synthetic tasks that do not reflect real-world complexity. Synthetic benchmarks present three fundamental limitations relative to real-world code. First, synthetic classes are self-contained with minimal dependencies, whereas real-world code requires framework integrations and project-specific patterns. Second, synthetic test suites rely on simplified assertions, whereas production code demands complex type systems and inheritance hierarchies. Third, synthetic benchmarks lack evolutionary complexity, since real-world classes must integrate with existing architectural patterns. These differences create a false sense of LLM capabilities because high performance on synthetic benchmarks provides no guarantee of success on realistic tasks, potentially misleading both researchers developing repository-level approaches and practitioners deploying LLM-based tools.

We address the issues above by curating a **new benchmark of real-world class-level code** from open-source projects, categorized as ‘**seen**’ (classes from repositories created before model training cutoffs, likely encountered during training) and ‘**unseen**’ (classes from repositories created after training cutoffs, guaranteeing they were not in training data). Our evaluation framework assesses LLM performance across different docstring variants, generation strategies, and RAG-augmented setups. We systematically analyze the impact of docstrings, the effectiveness of RAG, and the types of errors that arise in generated code, providing insights into the strengths and limitations of current LLMs. More specifically, we investigate the following research questions:

- **RQ1 (Real-World Performance): How do LLMs perform on class-level code generation in real-world projects compared to synthetic benchmarks?** We find that LLMs achieve 84–89% correctness on synthetic benchmarks but only 24–35% on real-world code. Models show nearly identical performance on seen and unseen real-world data.
- **RQ2 (Docstring Impact): Does the presence of docstrings improve the functional correctness of LLM-generated classes?** Our finding shows that complete docstrings provide improvements of 1–3% in correctness, in some cases, although this improvement is not frequent enough to gain statistical significance.
- **RQ3 (RAG for Unseen Code): Can retrieval-augmented generation (RAG) using seen data improve LLM performance on unseen class-level tasks?** We find that RAG provides substantial benefits of 4–7% specifically when documentation is incomplete (partial docstrings), but minimal value with complete documentation. This shows that RAG helps by providing concrete implementation examples when specifications lack implementation details.
- **RQ4 (Error Analysis): What are the most common errors in LLM-generated class-level code?** We find that `AttributeError`, `TypeError`, and `AssertionError` account for 84% of all failures. Synthetic benchmarks show assertion-dominated failures while real-world code shows attribute- and type-dominated failures, and RAG operates through error substitution, reducing logic failures while occasionally introducing dependency errors.

By answering the above RQs, this paper makes the following contributions:

1. A curated dataset of **real-world class-level code** from open-source projects, categorized as seen and unseen.
2. A systematic evaluation of LLM performance on class-level generation, comparing **real-world projects** to synthetic benchmarks.
3. An analysis of the impact of **docstrings** and **RAG** on functional correctness and error types.
4. Insights into **common errors** and practical implications for improving LLM-assisted programming tools.

The remainder of this paper is organized as follows: Section 2 discusses related work and background. Section 3 describes the design of our benchmark

and the overall view of the research methodology. Sections 4, 5, 6, and 7 address RQs 1, 2, 3, and 4, respectively. Section 8 discusses the implications of our findings. Section 9 describes some of the limitations of our study and outlines future directions. Section 10 discusses the threats to validity. Finally, Section 11 concludes the paper.

2 Background and Related Work

The evolution of code generation benchmarks follows a clear progression from function-level to repository-level tasks, yet a critical gap exists at the class-level. This section positions our contribution within this progression and establishes why class-level evaluation represents a necessary prerequisite for understanding repository-level capabilities.

2.1 Function-Level Code Generation

Large Language Models (LLMs) have achieved remarkable success in function-level code generation, enabling tools like GitHub Copilot and Amazon Code-Whisperer to assist developers (Chen et al., 2021; Austin et al., 2021). Established benchmarks like **HumanEval** (Chen et al., 2021) and **MBPP** (Austin et al., 2021) evaluate standalone function generation, while **CoderEval** (Yu et al., 2024) introduces non-standalone functions but maintains function-level granularity.

To understand current limitations, Wang et al. (Wang et al., 2025) analyzed 557 incorrect code snippets from six prominent LLMs on HumanEval, finding that syntactic errors account for 40% of failures, with a clear correlation between model size and error frequency. Liu et al. (Liu et al., 2023) introduced **EvalPlus**, which revealed that HumanEval’s test insufficiency allows models to pass with fragile code, reducing pass@k scores by 19.3–28.9% when comprehensive test suites are applied. These findings demonstrate that while LLMs excel at isolated function generation, they struggle with structural correctness and comprehensive test coverage.

2.2 Repository-Level Code Generation: The Emerging Frontier

Repository-level code generation has attracted growing attention as the ultimate goal for practical software engineering automation. Recent work addresses the complexities of multi-file, cross-dependency code generation through various approaches.

SWE-bench (Jimenez et al., 2023) introduced repository-level issue resolution, requiring models to understand and modify existing codebases to fix real GitHub issues. **HumanEvo** (Zheng et al., 2024) extends this with evolution-aware evaluation, tracking how repository structure changes over time. **CodeAgent** (Zhang et al., 2024) proposes tool-integrated agent systems

for repository-level coding challenges, demonstrating the need for sophisticated architectural scaffolding. For example, repository-level benchmarks like **RepoExec** (Le Hai et al., 2024) evaluate executability and cross-file context utilization, while surveys by Tao et al. (Tao et al., 2025) systematically categorize repository-level approaches and their limitations.

Recent advances continue pushing repository-level boundaries. Wu et al. (Wu et al., 2026) introduced **Environment-Aware Code Generation**, where generated code must be functionally correct and executable under arbitrary software configurations, revealing that current LLMs struggle with environment-specific requirements. **InlineCoder** (Hu et al., 2026) enhances repository understanding by inlining unfinished functions into call graphs, reframing repository challenges as function-level tasks. **CodeMEM** (Wang et al., 2026) addresses memory management in interactive repository-level generation through AST-guided dynamic context preservation.

2.3 Class-Level Code Generation: The Critical Gap

Between function-level and repository-level generation lies class-level code generation, which is a critical granularity that has received limited systematic evaluation. **ClassEval** (Du et al., 2023) represents the primary evaluation in this space, providing 100 manually crafted class-level tasks. However, its synthetic nature and small scale limit understanding of real-world class generation capabilities.

JavaBench (Cao et al., 2024) evaluates object-oriented code generation but focuses on project-level tasks requiring multiple interdependent classes rather than isolated class implementation, using data derived from university course projects rather than real-world software development. While valuable, this conflates class-level implementation capabilities with repository-level dependency resolution and does not reflect the complexity and patterns found in production codebases.

Recent concurrent work introduced **RepoClassBench** (Deshpande et al., 2024), which evaluates class-level generation from natural language descriptions within repository contexts. RepoClassBench emphasizes cross-file dependencies and repository-level reasoning, proposing the Retrieve-Repotools-Reflect (RRR) framework that equips LLMs with static analysis tools for iterative repository navigation. While RepoClassBench addresses repository-level complexity, it focuses on natural language-to-code generation with extensive tool augmentation rather than assessing fundamental class implementation capabilities from structured specifications.

2.4 Retrieval-Augmented Generation for Code

Retrieval-Augmented Generation (RAG) enhances LLM performance by incorporating relevant context during generation (Yang et al., 2025; Tao et al.,

2025). **CodeRAG-Bench** (Wang et al., 2024) systematically evaluated retrieval effectiveness across basic programming, open-domain, and repository-level tasks, revealing that while high-quality contexts improve generation, current retrieval models struggle with useful context fetching, particularly when queries lack lexical overlap with relevant documents.

CodeRAG (Li et al., 2025) addresses repository-level challenges by constructing dual graphs modelling functional dependencies and semantic relationships, achieving substantial improvements on repository-level benchmarks. However, these approaches assume successful implementation of individual components—an assumption our work tests by evaluating whether RAG helps at the fundamental class-level granularity.

2.5 Docstrings and Prompt Engineering

Documentation quality significantly impacts code generation performance (Yang et al., 2024; Ronanki et al., 2025). Yang et al. (Yang et al., 2024) demonstrated that strategic docstring compression (25–40% reduction) preserves generation quality while improving efficiency, revealing that not all documentation contributes equally. Taherkhani et al. (Taherkhani et al., 2024) showed that evolutionary prompt engineering achieves 6% improvement in pass@k while being 2–10 times more cost-effective than baselines.

However, existing work focuses on function-level tasks where docstrings describe single-purpose functionality. Class-level docstrings must describe complex object behaviours, method interactions, and state management—fundamentally different documentation challenges that require targeted evaluation.

2.6 Research Gap and Contribution

Current evaluation practices exhibit a critical discontinuity: function-level generation is well-studied, repository-level generation is rapidly advancing, but class-level generation, which connects these granularities, remains underexplored. This creates a fundamental knowledge gap about LLM capabilities.

The Class-Level Prerequisite Argument: Repository-level generation inherently requires successful class-level implementation. SWE-bench tasks often involve modifying or creating individual classes within the repository context. Coding agents’ tasks are not limited to the generation of functions only; they ultimately generate class implementations. Environment-aware generation must ensure classes execute correctly across configurations. Yet existing evaluations cannot isolate whether failures stem from class-level implementation deficiencies or repository-level context management issues.

This gap has practical implications: if LLMs cannot reliably implement individual classes given clear specifications, repository-level approaches may be addressing the wrong problem. Our work fills this gap by systematically evaluating class-level capabilities independent of repository context, providing

the foundation for understanding where repository-level approaches should focus their efforts.

Rather than competing with repository-level work, our contribution (detailed in Section 1) is to supply the prerequisite understanding it depends on: by isolating class-level capabilities from repository context, we enable precise diagnosis of where current LLMs struggle and where future repository-level approaches should focus their architectural innovations.

3 Methodology

3.1 Dataset Curation

To evaluate the performance of LLMs on real-world class-level code generation, we introduce *RealClassEval*, a dataset comprising two versions of Python classes: *pre-cutoff* and *post-cutoff*. Each version contains 200 classes mined from real-world GitHub open-source projects. The complete data curation process is illustrated in Figure 1, which shows our systematic approach from initial data collection through final dataset construction.

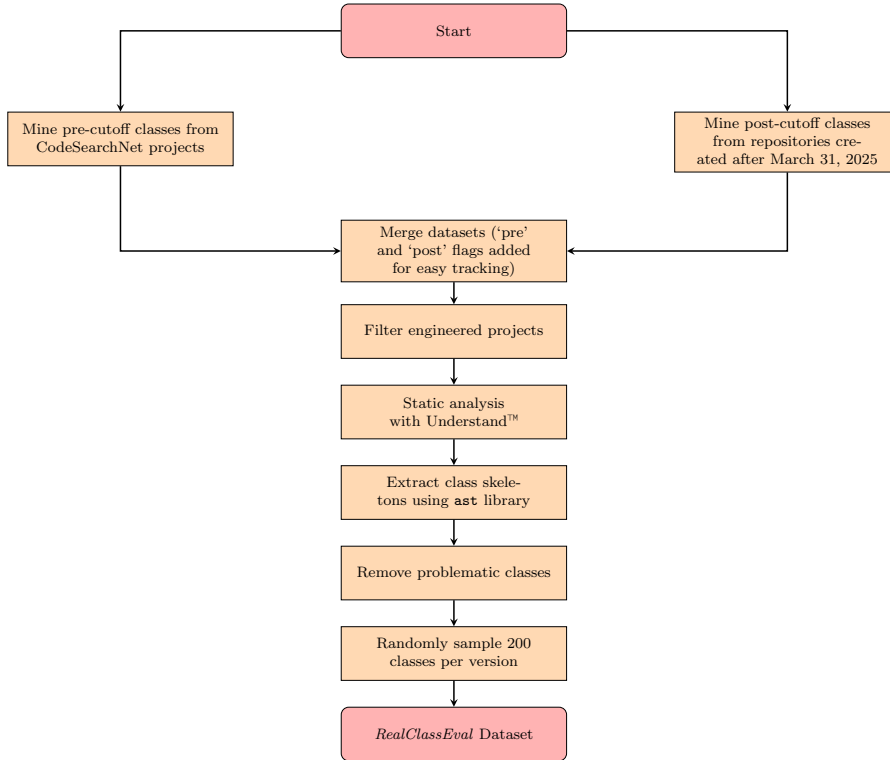


Fig. 1: Workflow for *RealClassEval* dataset curation.

3.1.1 Data Collection:

The *pre-cutoff* data is sourced from the **CodeSearchNet** dataset (Husain et al., 2019), which is one of the most widely used datasets for training code-related LLMs. **CodeSearchNet** has been used as a pre-training corpus for numerous prominent models, including StarCoder (Li et al., 2023), CodeT5+ (Wang et al., 2023), and UniXcoder (Guo et al., 2022), and has been used in numerous existing research papers (Jiang et al., 2024; Shi et al., 2023; Rahman et al., 2024; Orel et al., 2025). The dataset comprises 2 million (comment, code) pairs from open-source GitHub repositories across six programming languages. Even though we cannot guarantee that this dataset was indeed part of the training data of the LLMs we investigated in this research, we believe that it is a safe assumption to make, given the widespread adoption and popularity of this dataset in the LLM training community. The *post-cutoff* data consists of repositories created after March 31, 2025¹, guaranteeing these classes are unseen by the evaluated LLMs.

For the *post-cutoff* data collection, we used the GitHub search API to identify Python repositories created after March 31, 2025 systematically. We selected repositories that: (1) were created after our cutoff date to ensure temporal separation from LLM training data, (2) had active development with at least 10 commits to ensure maturity, (3) contained Python as the primary programming language, and (4) were not forks to eliminate duplicates. This systematic selection process yielded a pool of candidate repositories from which we subsequently applied filtering criteria to identify engineered projects.

3.1.2 Data Filtering

To ensure our dataset comprises high-quality, production-level software, we retain only *engineered projects* (Munaiah et al., 2017) from both the CodeSearchNet dataset and the post-cutoff repositories, adhering to the filtering criteria outlined in (Xiao et al., 2025). Importantly, we did not include all projects from CodeSearchNet; rather, we applied the same rigorous filtering criteria to both data sources to ensure consistency and quality. Specifically, we applied three filtering criteria to both *pre-cutoff* (CodeSearchNet) and *post-cutoff* (newly collected) repositories:

1. **License Declaration:** We excluded repositories not declaring a license or using non-standard licenses (marked as “Other” in the GitHub search tool). For the remaining repositories, we removed projects declaring licenses not commonly used for software projects, such as Creative Commons Attribution licenses and font licenses, as these typically indicate documentation or non-code projects.
2. **Project Size and Activity:** We filtered out repositories with fewer than a minimum threshold of stars (10+) and commits (50+) to ensure the

¹ All LLMs under investigation were either released before this date or their knowledge cut-off dates are before this date.

projects represent mature, community-recognized software rather than personal experiments or abandoned prototypes.

3. **Code Quality Indicators:** We excluded projects with incomplete documentation, lack of structured code organization (e.g., no clear module structure), or repositories that primarily contain tutorials, educational materials, or code snippets rather than production-quality software.

These filtering criteria, applied uniformly to both CodeSearchNet and post-cutoff repositories, ensure that our dataset comprises classes from well-maintained, professionally developed open-source projects that reflect real-world software engineering practices.

We report the concrete filtering funnel for both datasets in Appendix A.1 (Table 13).

3.1.3 Static Analysis and Class Extraction:

From the filtered engineered projects in both data sources, we use **UnderstandTM** (sci, 2024) to perform static analysis and identify Python classes. Class extraction is carried out using Python’s **ast** library, parsing all .py files to extract class skeletons while preserving class and method signatures and docstrings. An example of the class skeleton extraction process is shown in Listings 1 and 2. They illustrate how we transform a human-written class into a skeleton that serves as input for LLM code generation.

Listing 1: An example Python class from `codenerix/django-codenerix` (git, 2010).

```
class SubdomainMiddleware(object):
    """
    A middleware class that adds a 'subdomain' attribute to the
    current request.
    """
    def get_domain_for_request(self, request):
        """
        Returns the domain that will be used to identify the
        subdomain part
        for this request.
        """
        return get_domain()

    def process_request(self, request):
        """
        Adds a 'subdomain' attribute to the 'request'
        parameter.
        """
        domain, host = map(lower,
            (self.get_domain_for_request(request), request.
             get_host()))

        pattern = r'^(?: (?P<subdomain>.*?)\.)?%s(?:.*)?$' % re.
            escape(domain)
        matches = re.match(pattern, host)
```

```

if matches:
    request.subdomain = matches.group('subdomain')
else:
    request.subdomain = None
    logger.warning('The host %s does not belong to the
        domain %s, '
        'unable to identify the subdomain for this request
        ',
        request.get_host(), domain)

```

Listing 2: `Docstring` extracted from the class shown in Listing 1.

```

class SubdomainMiddleware(object):
    """
    A middleware class that adds a 'subdomain' attribute to the
    current request.
    """

    def get_domain_for_request(self, request):
        """
        Returns the domain that will be used to identify the
        subdomain part
        for this request.
        """
        pass

    def process_request(self, request):
        """
        Adds a 'subdomain' attribute to the 'request'
        parameter.
        """
        pass

```

3.1.4 Post-processing and Sampling

Post-processing involves removing problematic classes (e.g., those with Python 2 syntax), ensuring only parsable and valid skeletons remain. Following *ClassEval*’s exclusion criteria (Du et al., 2023), we omit tasks with complex execution dependencies, identified through static metrics. However, unlike *ClassEval*, we do not filter by domain, retaining a diverse range of topics such as Game Development, File Handling, and Management Systems. This approach ensures our dataset reflects the heterogeneity of real-world projects, providing a robust foundation for evaluating LLM performance on class-level code generation. From the remaining classes, we randomly sample 200 classes per version (*pre-cutoff* and *post-cutoff*) to create a balanced dataset for evaluation.

3.1.5 Dataset Comparability Validation

To ensure valid temporal comparisons and confirm that observed performance differences stem from model capabilities rather than dataset characteristics,

we conducted a comprehensive statistical analysis comparing pre-cutoff and post-cutoff distributions across key dimensions.

Our analysis reveals no statistically significant differences between the datasets across all measured characteristics (all p-values > 0.05 , Mann-Whitney U tests):

- **Code Complexity:** Both datasets show comparable structural complexity (lines of code: 74.3 ± 95.8 vs 187.8 ± 287.0 , $p=0.259$; methods per class: 2.5 ± 1.4 vs 3.7 ± 2.2 , $p=0.402$; cyclomatic complexity: 3.7 ± 5.0 vs 4.0 ± 4.5 , $p=0.699$).
- **Documentation Characteristics:** Documentation patterns are statistically indistinguishable (total docstrings: 3.4 ± 1.3 vs 4.5 ± 2.4 , $p=0.516$; comment-to-code ratio: 1.7 ± 2.6 vs 1.2 ± 1.8 , $p=0.383$).
- **Object-Oriented Features:** Class design characteristics show no significant differences (instance methods: 2.2 ± 1.6 vs 3.2 ± 2.5 , $p=0.368$; class coupling: 3.8 ± 5.9 vs 5.5 ± 6.2 , $p=0.224$).

While post-cutoff classes show numerically higher complexity values, these differences fall within expected random sampling variation and do not reach statistical significance under the tests applied. This comparability across structural, documentation, and design characteristics is consistent with our temporal split methodology and supports treating the pre-cutoff and post-cutoff datasets as comparable populations for experimental purposes. However, we acknowledge that the absence of a detected difference does not by itself establish equivalence. We treat this as a methodological assumption, not a proven guarantee.

These comparable distributions ensure that our seen versus unseen comparisons (RQ1), docstring effects experiments (RQ2), retrieval-augmented generation experiments (RQ3), and error pattern analyses (RQ4) reflect genuine model capabilities rather than confounding dataset characteristics. The statistical equivalence of our temporal splits provides a strong methodological foundation for attributing observed performance differences to the experimental factors under investigation.

3.2 Code Generation

3.2.1 Selected LLMs

To evaluate the performance of LLMs on class-level code generation, we selected seven state-of-the-art models: **Qwen 2.5 Coder** (Hui et al., 2024), **GPT-4.1** (Ope, 2025a), **GPT-5** (Ope, 2025b), **GPT-OSS** (Agarwal et al., 2025), **Codestral** (mis, 2024), **DeepSeek-V3** (Liu et al., 2024), and **Llama-4 Maverick** (met, 2025). These models were chosen to represent a diverse range of architectures, training paradigms, and capabilities, ensuring a comprehensive evaluation of both general-purpose and code-specific LLMs, as well as models with varying reasoning abilities. They also vary in their release dates,

Table 1: Characteristics of the evaluated LLMs.

Model	Release Date	Knowledge Cutoff	Parameters	Key Characteristics
Qwen 2.5 Coder	September 2024	June 2024	32B	Code-specific, optimized for code generation and reasoning
GPT-4.1	April 2025	June 2024	1.8T (estimated)	General-purpose, advanced reasoning and instruction-following
GPT-5	August 2025	October 2024	2.5T (estimated)	General-purpose, state-of-the-art reasoning and context handling
GPT-OSS	August 2025	June 2024	20B	General-purpose (open), strong code capabilities
Codestral	January 2025 (Latest)	Unknown	100B	Code-specific, specialized for code, lightweight and efficient
DeepSeek-V3	August 2025	July 2024	126B	General-purpose, focused on long-context code understanding
Llama-4 Maverick	April 2025	August 2024	400B	General-purpose, balanced reasoning and code generation

knowledge cutoffs, parameter sizes, context window lengths, and specialization. Table 1 summarizes these characteristics and highlights the diversity of our evaluation set.

3.2.2 Prompt Design

We use a standardized prompt to ensure consistency across all models. The prompt is designed to instruct the LLM to implement a complete Python class based on the provided class skeleton, without additional explanations. The prompt template is as follows:

You are an expert Python programmer who can correctly implement complete Python classes based on the provided class skeletons. Implement the following class. Do not explain the code. The given class skeleton is as follows:
[CLASS SKELETON]

Unlike *ClassEval* (Du et al., 2023), which explores three generation strategies (holistic, incremental, and interactive), we focus exclusively on **holistic generation**. This decision is motivated by the findings of ClassEval, which indicate that models with longer context windows (such as GPT-3.5 and GPT-4 in their study) perform well in holistic generation. Given that all our selected models have substantial context windows (ranging from 128K to 1M tokens), we opt for holistic generation to leverage their ability to process and generate entire class implementations in a single pass.

3.3 Test Suite Generation

To assess functional correctness, we use **PYNGUIN** (Lukasczyk and Fraser, 2022) to automatically generate test suites for the classes in our dataset. **PYNGUIN** (the PYthoN General UnIt test geNerator) is an automated unit test generation framework specifically designed for Python programs. Unlike test generation tools for statically typed languages like Java, **PYNGUIN** addresses the unique challenges posed by Python’s dynamic typing and lack of explicit type information. The tool takes a Python module as input along with

its dependencies and automatically generates unit tests that maximize code coverage using various test generation algorithms.

We specifically employ PYNGUIN’s **DynaMOSA** (Dynamic Many-Objective Sorting Algorithm) (Panichella et al., 2017) algorithm for test generation. **DynaMOSA** is a many-objective evolutionary algorithm specifically designed for automated test case generation with dynamic selection of coverage targets. The algorithm treats test case generation as a many-objective optimization problem where each coverage target (e.g., branches, statements) is treated as a separate objective. **DynaMOSA** dynamically adjusts its focus based on the control dependency hierarchy, prioritizing uncovered targets and progressively exploring the search space. In the original paper that introduced PYNGUIN, DynaMOSA achieved 68.0% mean branch coverage across their benchmark, outperforming other search-based algorithms (MIO: 67.0%, MOSA: 67.8%) and significantly exceeding random generation (63.6%), establishing it as the most effective approach for Python test generation (Lukasczyk and Fraser, 2022).

PYNGUIN invocation parameters. We used PYNGUIN version 0.40.0 with the following configuration, applied identically to every class: maximum iterations 100; maximum search time 180 seconds per class (whichever limit is reached first terminates the search); assertion generation mode **SIMPLE**; and a single fixed random seed, 175, applied uniformly across all classes. Branch coverage was extracted directly from PYNGUIN’s own per-run coverage report.

3.3.1 Coverage Analysis and Methodological Validation

Our coverage analysis reveals that PYNGUIN achieves markedly different coverage patterns across synthetic versus real-world code: 93.2% average branch coverage on ClassEval’s manually crafted classes, versus 58.7% on *RealClassEval* classes. Both figures are consistent with DynaMOSA’s reported 68.0% mean coverage in its original evaluation (Lukasczyk and Fraser, 2022), and the differential tracks the added structural complexity of real-world code, which includes deeper conditional logic, broader error handling, and external integration points, relative to synthetic benchmarks’ simplified control flow. This pattern indicates PYNGUIN is sensitive to the same implementation complexity this study investigates. We discuss the measurement limitations this coverage gap introduces in Section 10. To assess whether generated test suites include explicit correctness checks rather than relying solely on implicit exception-based checks, we sampled $n = 78$ *RealClassEval* classes – the size required for a 10% margin of error at 95% confidence given our population of 400 real-world classes (Cochran, 1977). These classes were allocated proportionally across a 2×2 stratification by pre-cutoff/post-cutoff origin and pass/fail outcome (25 pre-cutoff-fail, 12 pre-cutoff-pass, 27 post-cutoff-fail, 14 post-cutoff-pass). We coded each class’s generated test suite as *assertion-based* (containing at least one explicit `assert` statement, whether checking type, state, or return values) or *assertion-free* (relying solely on `pytest.mark.xfail`-decorated calls that

implicitly expect an exception without any explicit check). 69 of 78 (88.5%) test suites were assertion-based; 9 (11.5%) were assertion-free. This assessment establishes that the large majority of generated test suites contain explicit assertions, addressing whether tests check anything beyond exception-free execution. It does not, however, establish that those assertions correctly capture the intended semantics of the original class, or how effectively the suites would detect functional faults (e.g., via mutation testing). We treat semantic and fault-detection adequacy of the automatically generated oracle as an open limitation, discussed further in Section 9.

We adopt PYNGUIN over alternative test-construction approaches for scalability and consistency. Building executable environments for hundreds of unmaintained real-world repositories to run their native test suites is intractable at our scale. Manual test authoring for 400 diverse classes would similarly require months of expert effort while introducing subjective bias. PYNGUIN instead applies identical algorithmic parameters across all classes and models, enabling directly comparable evaluation across the full dataset without manual intervention.

3.4 Result Analysis

3.4.1 Performance Analysis

To evaluate the performance of LLMs on class-level code generation, we employ **pass rate** as our primary metric, defined as the ratio of passed tests to the total number of tests for each generated class. This metric is directly related to the widely used **Pass@1** (Chen et al., 2021), which measures the probability that a generated solution passes all tests in a single attempt. While **Pass@1** is typically used for function-level evaluation, our pass rate extends this concept to class-level correctness, providing a robust measure of functional accuracy across the entire class implementation. For a given class i , the pass rate metric is defined as:

$$\text{Pass Rate}_i = \frac{\text{Number of Passed Tests}_i}{\text{Total Number of Tests}_i} \quad (1)$$

We compare one pass rate distribution against another to see:

- if a statistically significant difference exists between distributions.
- when a statistically significant difference exists, how big the difference is in terms of effect size.

We detail the statistical approaches taken to answer the RQs in their respective sections.

3.4.2 Error Analysis:

We analyze the distribution of error types (e.g., `AttributeError`, `TypeError`) across data splits, docstring conditions, and RAG settings. The distributions

are used to show the overall error patterns. Then we go deeper into the analysis by comparing the distributions against one another to see if they differ with enough statistical significance.

In summary, by focusing on pass rates, paired comparisons, and error distributions, we provide a comprehensive and reproducible assessment of factors influencing functional correctness in real-world settings.

4 Real-World Performance

This section addresses RQ1: *How do Large Language Models (LLMs) perform on class-level code generation in real-world projects compared to synthetic benchmarks?* We evaluate all seven LLMs (Codestral, Deepseek-V3, GPT-4.1, GPT-5, GPT-OSS, Llama 4 Maverick, Qwen Coder 2.5) across three datasets: *ClassEval* (100 synthetic classes), *RealClassEval* pre-cutoff (200 classes from CodeSearchNet, likely seen during training), and *RealClassEval* post-cutoff (200 classes from post-cutoff projects, unseen). The performance metric is the pass rate, defined as the proportion of passed test cases per class.

4.1 Approach

To compare LLM performance across synthetic and real-world datasets, we employed a multi-step statistical approach combining per-class and pooled analyses.

4.1.1 Data Processing

We processed test reports for each LLM per dataset. Per-class pass rates were computed for analysis. For the pooled analysis, test cases were aggregated into total passed and failed counts per dataset and LLM.

4.1.2 Kruskal-Wallis Test

We used the Kruskal-Wallis test (Kruskal and Wallis, 1952) to compare per-class pass rates across the three datasets (ClassEval, pre-cutoff, post-cutoff) for each LLM. In the current context, the null hypothesis states that the pass rate distributions are identical across datasets, while the alternative hypothesis states that at least one dataset’s distribution differs.

$$H_0 : F_{ClassEval} = F_{pre-cutoff} = F_{post-cutoff} \quad (2)$$

$$H_A : F_j \neq F_k \text{ for at least one pair } (j, k), \\ \text{where } j, k \in \{\text{ClassEval, pre-cutoff, post-cutoff}\}, j \neq k \quad (3)$$

The Kruskal-Wallis test is a non-parametric alternative to one-way ANOVA, suitable for non-normal data and independent samples across three or more groups. The non-normality of pass rates was confirmed by the Shapiro-Wilk test (Shapiro and Wilk, 1965), justifying the non-parametric approach.

4.1.3 Mann-Whitney U Tests

For significant Kruskal-Wallis results, we conducted pairwise Mann-Whitney U tests (Mann and Whitney, 1947) (ClassEval vs. pre-cutoff, ClassEval vs. post-cutoff, pre-cutoff vs. post-cutoff) to identify specific differences. The Mann-Whitney U test is a nonparametric test suitable for unpaired samples (i.e., distinct classes across datasets) and non-normal distributions. We applied Benjamini-Hochberg (Benjamini and Hochberg, 1995) False Discovery Rate (FDR) correction globally across all 21 tests (3 comparisons per LLM, 7 LLMs) to control for multiple comparisons. The pairwise tests pinpoint which datasets differ, while FDR correction balances Type I error across all tests.

4.1.4 Effect Sizes and Confidence Intervals

We computed Cliff’s Delta (Cliff, 1993; Macbeth et al., 2011) to quantify the magnitude of pairwise differences, with thresholds (Cohen, 2013; Rahman et al., 2024): negligible (< 0.147), small (< 0.33), medium (< 0.474), large (≥ 0.474). Bootstrap 95% confidence intervals (CIs) were calculated for Cliff’s Delta to quantify uncertainty, using 1,000 iterations. Cliff’s Delta provides an effect size for ordinal data, and CIs ensure robust estimation of differences. Although Cliff’s Delta help us get the effect size of differences, it does not quantify the actual differences. Hence, we also calculate the mean differences between the distributions with bootstrap 95% CIs for an easy interpretation.

4.1.5 Pooled Analysis

We performed a chi-square analysis (Pearson, 1900; Fisher, 1922) on pooled test case counts (passed vs. failed) using 2×2 contingency tables for pairwise dataset comparisons. The chi-square test is appropriate for categorical outcomes. Cramér’s V (Cramér, 1999) quantified effect sizes (Cohen, 2013): negligible (< 0.1), small (< 0.3), medium (< 0.5), large (≥ 0.5), with bootstrap CIs for proportion differences. FDR correction was applied globally across all 21 tests. The pooled analysis provides an aggregate view of performance, complementing per-class results and ensuring robustness.

4.2 Findings

The analysis revealed significant performance differences across datasets, with *ClassEval* consistently outperforming both versions of *RealClassEval*. Figure 2 shows the overall trend of performance of different LLMs across the three

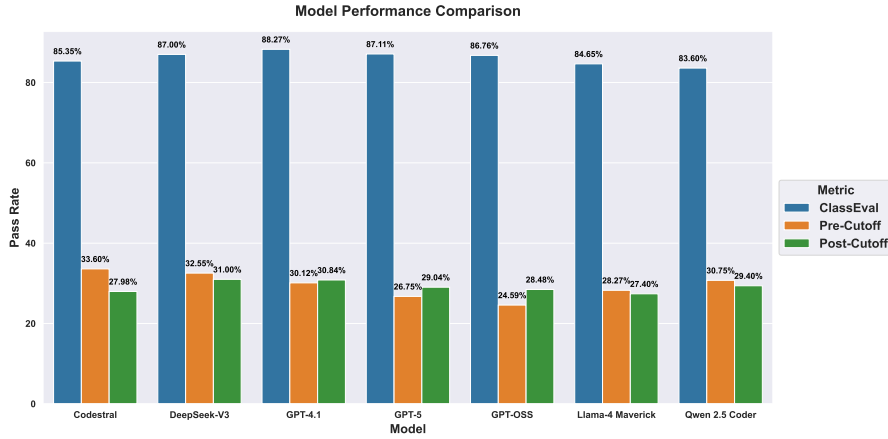


Fig. 2: Performance of different LLMs on ClassEval, Pre-Cutoff, and Post-Cutoff data.

Table 2: Per-Class Pass Rate Analysis, condensed across all seven models (full per-model breakdown in Table 14, Appendix A.2).

Comparison	Mean Diff. Range (95% CI)	Effect Size Range	# Models Significant / 7
ClassEval vs. Pre-Cutoff	46.93–61.24%	Medium–Large	7 / 7
ClassEval vs. Post-Cutoff	43.96–57.05%	Medium–Large	7 / 7
Pre-Cutoff vs. Post-Cutoff	–9.05–4.57%	Negligible	1 / 7 (GPT-OSS)

datasets under investigation, while Tables 2 and 3 summarize the per-class analysis result and pooled analysis result, respectively.

4.2.1 Per-Class Analysis

The Kruskal-Wallis test revealed significant differences in per-class pass rates across datasets for all LLMs (all $p < 1 \times 10^{-14}$). Pairwise Mann-Whitney U tests (Table 2) confirmed significant differences for ClassEval vs. pre-cutoff and ClassEval vs. post-cutoff, with mean pass rate differences of 46.93–61.24% and 43.96–57.05%, respectively, and medium to large effect sizes (Cliff’s Delta between 0.46 and 0.64). pre-cutoff vs. post-cutoff comparisons showed no significant differences for six of seven models after FDR correction. GPT-OSS showed a statistically significant difference (FDR-adjusted $p = 0.019$), though the effect size remained negligible (Cliff’s Delta = -0.125). All pre-cutoff vs. post-cutoff comparisons had negligible practical effect sizes. Power analysis across all seven models indicated high statistical power (0.974–0.983), ensuring reliable detection of differences.

Table 3: Pooled Chi-Square Analysis, condensed across all seven models (full per-model breakdown in Table 15, Appendix A.2).

Comparison	Mean Diff. Range (95% CI)	Effect Size Range (Cramér’s V)	# Models Significant / 7
ClassEval vs. Pre-Cutoff	51.75–62.17%	Medium-Large	7 / 7
ClassEval vs. Post-Cutoff	54.20–58.28%	Large	7 / 7
Pre-Cutoff vs. Post-Cutoff	–3.89–5.62%	Negligible	0 / 7 (all FDR-adj. $p > 0.10$)

4.2.2 Pooled Analysis

The chi-square analysis (Table 3) on pooled test case counts confirmed significant differences for ClassEval vs. pre-cutoff and ClassEval vs. post-cutoff, with pass rate differences of 51.75–62.17% and 54.20–58.28%, respectively, and medium to large effect sizes (Cramér’s V ranging between 0.44 to 0.56). No significant differences were found for pre-cutoff vs. post-cutoff across all models (all FDR-adjusted $p > 0.07$, negligible effect sizes). Bootstrap 95% CIs confirmed that the observed gaps are robust.

4.3 Interpretation

The findings indicate that LLMs perform significantly better on synthetic benchmarks (ClassEval, 83.60–88.27%) than real-world projects (pre-cutoff and post-cutoff, 24.59–33.60%), answering RQ1. pre-cutoff and post-cutoff classes showed no significant performance difference for six of seven models, with negligible effect sizes across all models. It suggests that we find no evidence that corpus-level familiarity with related code patterns meaningfully improves real-world class-level code generation, rather than a direct test of memorization per se. This distinction matters because CodeSearchNet is a function-level corpus with (comment, code) pairs, not a corpus of complete classes. Even where a model’s training data overlapped with CodeSearchNet, it would have encountered the constituent functions individually rather than the class-level artifacts we evaluate here, and reconstructed as such in our dataset. Our seen versus unseen split should therefore be read as an approximation of corpus-level exposure to related code, not a confirmed measure of exposure to the specific evaluation artifacts, and we discuss this construct-validity limitation further in Section 9. Our results further show that the real-world performance range (24.59–33.60%) is directionally consistent with the recent work called RepoClassBench (Deshpande et al., 2024), which reports 24–34% baseline performance on Python classes using natural language descriptions and repository context rather than just class skeletons. We note this alignment as suggestive rather than conclusive, since the two studies use non-comparable evaluation instruments (our PYNGUIN-generated automated tests versus RepoClassBench’s repository-native test suites) and differ in task setup. We therefore do not treat this as a controlled comparison capable of supporting strong claims that specification richness generally overcomes implementation limitations.

The high pass rate of *ClassEval* compared to *RealClassEval* was significant, but the reasons were initially unclear, as the large effect sizes (Cliff’s Delta 0.46–0.64) and pass rate differences did not fully explain the gap. To investigate, we conducted a mixed-effects model (Laird and Ware, 1982; Carey and Wang, 2001) analysis, including dataset, test count, and LLM-generated code metrics (lines of code, method count, cyclomatic complexity, and class coupling) as predictors, with LLM as a random effect. We chose these 4 static metrics because they represent both the length of the code as well as the complexity of the code. The model showed significant dataset effects (e.g., $\beta = -1.060$, $p < 0.001$) and a negative test count effect ($\beta = -0.014$, $p < 0.001$). While lines of code ($\beta = -0.002$, $p = 0.006$) and cyclomatic complexity ($\beta = 0.043$, $p < 0.001$) showed statistical significance, their effects were small relative to the dataset effect, indicating that code complexity metrics explain only a minor portion of the performance gap.

To test whether the synthetic-versus-real-world gap reflects genuine implementation difficulty rather than dependency exposure, we split classes into context-free and dependency-heavy subgroups by class coupling and re-fit the mixed-effects model within each. The dataset effect persisted at comparable magnitude in both subgroups ($\beta = -0.90$ to -0.86 context-free; $\beta = -1.10$ to -0.95 dependency-heavy; all $p < 0.001$). This indicates the gap is not primarily an artifact of dependency exposure or class coupling. If anything, the gap is slightly larger among dependency-heavy classes, suggesting real-world implementation difficulty extends beyond what coupling alone would predict.

We further tested whether the gap is attributable to differential test-suite coverage by adding branch coverage as a covariate to the same model on classes with available per-class coverage data. The dataset effect was modestly attenuated but remained large and highly significant with coverage included ($\beta = -0.902$ vs. -0.940 for *ClassEval*–pre-cutoff; $\beta = -0.950$ vs. -0.976 for *ClassEval*–post-cutoff, with vs. without coverage; all $p < 0.001$), and coverage itself was a significant predictor ($\beta = 0.149$, $p = 0.001$). Together with the subgroup analysis above, this indicates the synthetic-versus-real-world performance gap is not primarily attributable to the confounds we can directly measure within our design: dependency exposure, class coupling, or test-suite coverage differences. This analysis cannot, however, rule out the confound shared by every class in our evaluation: the complete absence of repository-level context. The evidence therefore strongly supports that real-world, context-free class generation remains challenging for current LLMs, though it does not fully separate class-level implementation difficulty from the effects of missing project-specific context.

We then qualitatively analyzed test suites for randomly selected classes. The analysis revealed that *ClassEval* test suites used simple equality assertions (e.g., numerical, dictionary checks) with no external dependencies, while *RealClassEval* suites involved complex type metadata checks and system dependencies (e.g., `numpy`, `MCPCContext`), leading to errors like `AttributeError`. This suggests that *ClassEval*’s simpler test suites, despite higher test counts,

enable higher pass rates by requiring straightforward functionality, unlike *RealClassEval*’s complex, dependency-heavy tests.

RQ1 Findings: LLMs achieve significantly higher pass rates on synthetic benchmarks compared to real-world projects. Six of seven models show no significant performance differences between pre-cutoff and post-cutoff real-world data, with all models exhibiting negligible effect sizes. We find no evidence that corpus-level exposure to related code patterns meaningfully improves real-world class-level performance, though we cannot rule out post-training exposure, distillation, or other contamination pathways for closed models, and note that CodeSearchNet’s function-level granularity limits how directly this speaks to memorization of class-level artifacts specifically.

5 Impact of Docstrings

To understand the impact of documentation on code generation quality, we conducted a comprehensive ablation study examining how varying levels of docstring completeness affect the functional correctness of LLM-generated classes. This section addresses RQ2: *Does the presence of docstrings improve the functional correctness of LLM-generated classes?* We evaluate seven LLMs (Codestral, Deepseek-V3, GPT-4.1, GPT-5, GPT-OSS, Llama 4 Maverick, Qwen Coder 2.5) across two versions of *RealClassEval*: pre-cutoff and post-cutoff. Like before, the performance metric is the pass rate, defined as the proportion of passed test cases per class.

5.1 Approach

To assess the effect of docstring completeness on LLM performance, we employed a within-subjects experimental design combining per-class and pooled analyses. The per-class analysis preserves granularity by comparing pass rates across docstring conditions, while the pooled analysis aggregates test case outcomes to assess overall trends. The following details the steps.

5.1.1 The Ablation Study

We evaluated all seven LLMs across three documentation conditions to isolate the impact of documentation completeness on class-level code generation. This systematic ablation study controls for snippet-specific characteristics by testing each class under all documentation conditions, enabling precise measurement of documentation effects.

Our experimental design implements three documentation levels that reflect realistic development scenarios: **(1) full docstrings** with complete documentation for all program units, including comprehensive class descriptions,

method purposes, parameter specifications, and return value descriptions; **(2) partial docstrings** with documentation for a randomly selected subset of program units, simulating incomplete coverage common in real projects; and **(3) no docstrings** providing only function signatures and class structure.

The partial docstrings condition represents the most realistic scenario in software development, created by systematically removing docstrings from the full condition through random selection. We ensured that at least one documented component remained per class while varying coverage patterns. This process yielded partial documentation that retained an average of 60% of the original content (mean: 127 tokens vs. 203 tokens for full condition), closely mimicking incomplete documentation found in production codebases.

To illustrate this methodology, we demonstrate the ablation process using the skeleton extracted from the `SubdomainMiddleware` class shown in Listing 2. The full docstrings condition includes complete documentation as extracted from the original implementation. From this baseline, our random removal process eliminates selected docstrings to create the partial condition, as shown in Listing 3:

Listing 3: Partial docstrings condition - random removal of some docstrings from `SubdomainMiddleware`.

```
class SubdomainMiddleware(object):
    def get_domain_for_request(self, request):
        pass

    def process_request(self, request):
        """
        Adds a "subdomain" attribute to the 'request' parameter.
        """
        pass
```

In this example, the class-level docstring along with the first method docstring were removed while the second method docstring was retained, creating an incomplete specification that tests whether LLMs can infer the missing class purpose from available method documentation. The no docstrings condition removes all documentation, as shown in Listing 4:

Listing 4: No docstrings condition - only structural information for `SubdomainMiddleware`.

```
class SubdomainMiddleware(object):
    def get_domain_for_request(self, request):
        pass

    def process_request(self, request):
        pass
```

This systematic progression from complete to absent documentation enables precise measurement of how information availability affects generation quality. Each class in our dataset was evaluated under all three conditions, yielding paired samples that control for class-specific complexity while isolating documentation effects for repeated-measures statistical analysis.

5.1.2 Friedman Test

Given the continuous and bounded nature of our outcome variable (pass rate $\in [0, 1]$) and the paired structure of our data, we employed non-parametric statistical methods. The Friedman test (Friedman, 1937, 1940) compared pass rates across three docstring conditions (full, partial, no) for each model-dataset combination, testing the null hypothesis that pass rate distributions are identical versus the alternative that at least one condition differs.

5.1.3 Wilcoxon Signed Rank Test

For significant Friedman results, post-hoc Wilcoxon signed-rank tests (Wilcoxon, 1992) assessed pairwise differences (full vs. partial, full vs. no). High skewness ($|skewness| > 1$ in 11 of 28 comparisons) prompted sign tests to validate Wilcoxon results.

Similar to RQ1, other steps of the analysis include a pooled chi-square analysis on aggregated pass-fail counts, using 2×2 contingency tables for pairwise comparisons. Benjamini-Hochberg False Discovery Rate (FDR) correction ($\alpha = 0.05$) was applied separately per dataset (7 Friedman, 14 Wilcoxon, 14 chi-square tests per dataset), treating pre-cutoff and post-cutoff as independent experiments on different code populations. Effect sizes included Cliff's Delta (negligible < 0.147 , small < 0.33 , medium < 0.474 , large ≥ 0.474) and Cramér's V (negligible < 0.1 , small < 0.3 , medium < 0.5 , large ≥ 0.5), with bootstrap 95% confidence intervals (CIs).

5.1.4 Temporal Analysis

A temporal analysis computed benefit ratios ($\# \text{ improved} / \# \text{ worsened}$) and net benefits ($\# \text{ improved} / \# \text{ unchanged} \times 100$) for pre-cutoff and post-cutoff datasets, with a Wilcoxon test comparing model-wise ratios to assess whether docstring benefits differ between seen and unseen data.

5.2 Findings

The analysis revealed limited and model-specific effects of docstrings on code generation quality. Figure 3 shows pass rate distributions across docstring conditions, with only subtle differences between conditions. Tables 4, 5, and 6 summarize per-class, model-specific, and temporal results, respectively.

5.2.1 Per-Class Analysis

The Friedman test revealed significant differences in pass rates across docstring conditions for three model-dataset combinations after FDR correction: Codestral (pre-cutoff: $p = 1.02e - 03$, $p_{\text{FDR}} = 0.007$), GPT-4.1 (pre-cutoff: $p = 2.93e - 03$, $p_{\text{FDR}} = 0.010$), and Deepseek-V3 (post-cutoff: $p = 6.81e - 03$,

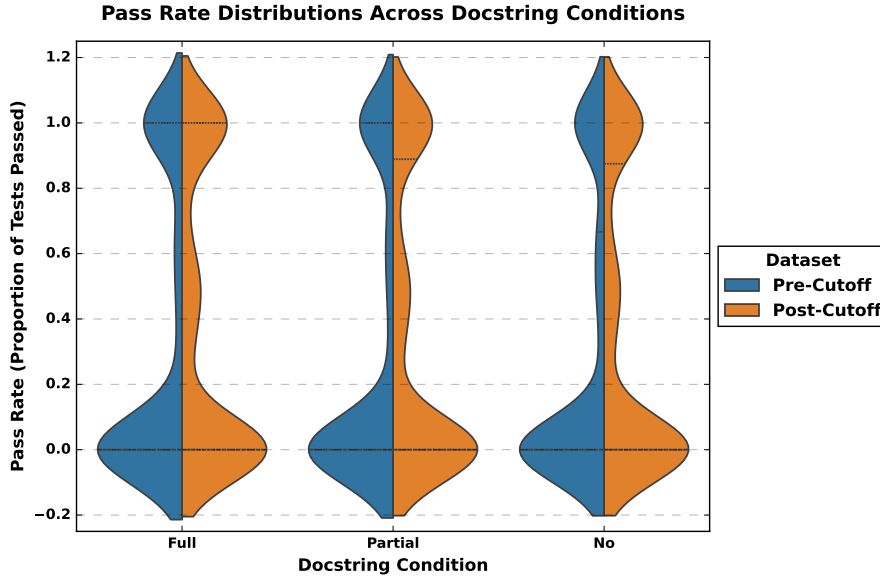


Fig. 3: Violin plot of per-class pass rate distributions across docstring conditions (full, partial, no) for Pre-Cutoff and Post-Cutoff datasets, aggregated across all models. Violin plots use kernel density estimation to show data distribution; the y-axis extends beyond $[0,1]$ due to KDE smoothing (Silverman, 2018), though all actual pass rate values are bounded between 0 and 1.

$p_{\text{FDR}} = 0.048$). However, pairwise Wilcoxon tests (Table 4) showed only two significant improvements after FDR correction: Codestral (pre-cutoff, full vs. partial: $p_{\text{FDR}} = 0.049$) and Deepseek-V3 (post-cutoff, full vs. partial: $p_{\text{FDR}} = 0.048$), with mean differences of -0.058 (-2.72%) and -0.047 (-1.51%) tests, respectively. The negative values indicate that full docstrings outperform partial docstrings. Notably, the full vs. no comparison yielded no significant results after FDR correction.

All Cliff’s Delta values ranged from 0.003 to 0.077 (negligible effect sizes), reflecting minimal practical magnitudes. High skewness prompted sign tests for 11 comparisons, with two showing significance after FDR correction, corroborating the corresponding Wilcoxon results. The remaining 26 of 28 Wilcoxon comparisons showed no significant effects, indicating that for most models, docstrings do not significantly impact functional correctness.

5.2.2 Pooled Analysis

The chi-square analysis on pooled test case counts showed no significant differences after FDR correction for either dataset (all $p_{\text{FDR}} > 0.05$). Cramér’s V values ranged from 0.002 to 0.066 (negligible effect sizes), confirming that aggregate test-level effects are minimal. This finding suggests that while indi-

Table 4: Summary of docstring ablation results across all models and datasets with respect to the number of test cases. Negative differences indicate full docstrings cause more test cases to pass compared to the alternative condition.

Comparison	Mean % Diff Range	Improved	Worsened	Unchanged	Significant (after FDR Correction)	Effect Size
Full vs. Partial	-0.15% to -2.72%	97	147	2,371	2/28	Negligible
Full vs. No	-0.44% to -3.13%	113	235	2,267	0/28	Negligible
Combined	-0.15% to -3.13%	210	382	4,638	2/56	Negligible

Table 5: Model-specific performance improvements with full docstrings. Percentages represent pass rate increases when using full documentation relative to partial or no docstrings. **Bold** entries achieved statistical significance after FDR correction ($p_{\text{FDR}} < 0.05$).

Model	Dataset	Full vs. Partial (%)	Full vs. No (%)	Average Benefit (%)
Codestral	Pre-Cutoff	2.72	3.13	2.93
	Post-Cutoff	1.01	0.80	0.90
Deepseek-V3	Pre-Cutoff	0.93	1.59	1.26
	Post-Cutoff	1.51	1.32	1.42
GPT-4.1	Pre-Cutoff	0.51	2.11	1.31
	Post-Cutoff	1.02	0.90	0.96
GPT-5	Pre-Cutoff	0.71	1.59	1.15
	Post-Cutoff	0.15	0.93	0.54
GPT-OSS	Pre-Cutoff	1.33	1.58	1.45
	Post-Cutoff	0.32	0.63	0.48
Llama 4 Maverick	Pre-Cutoff	1.88	2.58	2.23
	Post-Cutoff	0.64	0.44	0.54
Qwen Coder 2.5	Pre-Cutoff	0.41	1.91	1.16
	Post-Cutoff	0.93	1.36	1.15

vidual snippets may show improvements, these effects do not accumulate to produce population-level differences in test case outcomes.

5.2.3 Model-Specific Analysis

Table 5 presents model-specific improvements with full docstrings. Only two of 14 model-dataset combinations showed statistically significant benefits: Codestral in pre-cutoff data (full vs. partial: 2.72%, $p_{\text{FDR}}=0.049$) and Deepseek-V3 in post-cutoff data (full vs. partial: 1.51%, $p_{\text{FDR}}=0.048$). The remaining 12 combinations showed no significant effects despite observed numerical differences ranging from 0.15% to 3.13%.

5.2.4 Temporal Analysis

Table 6 presents the comparative impact of docstrings across temporal boundaries. The benefit ratio ($\#$ improved / $\#$ worsened) was 1.58:1 for pre-cutoff data and 2.14:1 for post-cutoff data, representing a numerical difference of 35%. However, a Wilcoxon test comparing model-wise benefit ratios was non-significant ($p=0.326$), indicating no statistically reliable difference in docstring effectiveness between pre-cutoff and post-cutoff data. This null result reflects

Table 6: Temporal analysis of docstring impact. Benefit ratio is the ratio of classes that improved to classes that worsened with full docstrings. The Wilcoxon test comparing Pre-Cutoff vs. Post-Cutoff benefit ratios across models was non-significant.

Dataset	Classes Helped	Classes Hurt	Benefit Ratio	Net Benefit (%)
Pre-Cutoff	210 (8.9%)	133 (5.6%)	1.58:1	3.3%
Post-Cutoff	172 (7.1%)	80 (3.3%)	2.14:1	3.8%

substantial model-specific variability: Codestral showed stronger post-cutoff benefits (benefit ratio 0.61 post vs. 0.30 pre for full vs. no), while Deepseek-V3 showed the opposite pattern (0.37 post vs. 0.60 pre). The lack of a consistent temporal effect suggests that docstring effectiveness depends primarily on model-specific factors rather than whether code was in the training data.

5.3 Interpretation

The findings provide limited evidence that docstrings improve LLM code generation quality, addressing RQ2 with a nuanced answer: **for most models, docstrings do not significantly improve functional correctness**. Only 2 of 7 models (Codestral and Deepseek-V3) showed statistically significant benefits in specific conditions, and even these effects were negligible in magnitude. This pattern, combined with adequate statistical power (0.78–0.87) obtained from power analysis, suggests that docstring effects are genuinely minimal rather than obscured by insufficient sample sizes.

5.3.1 Interpreting Limited Effects

The lack of widespread significance after FDR correction indicates that raw numerical improvements (averaging 1.25% across all comparisons) are largely attributable to random variation and multiple testing rather than true effects. The fact that pooled chi-square tests found zero significant results reinforces this interpretation: individual snippet-level fluctuations do not aggregate into reliable population-level patterns.

The high skewness in 11 of 28 comparisons reveals that most classes show zero difference across docstring conditions, with occasional large outliers driving mean differences. This suggests docstring benefits may be highly context-dependent, helping only in specific scenarios rather than providing universal improvements. The negligible Cliff’s Delta values (all < 0.15) confirm minimal practical effect sizes, even for the two statistically significant findings.

5.3.2 Model-Specific Heterogeneity

The restriction of significant effects to two models (Codestral and Deepseek-V3) suggests that docstring processing may depend on model architecture or training procedures. Possible explanations include:

1. **Training data composition:** Models trained on repositories with comprehensive docstring conventions may learn stronger associations between documentation and correct implementations.
2. **Attention mechanisms:** Architectural differences in how models weight docstring tokens versus signature tokens could affect documentation utilization.
3. **Model capacity:** Smaller or more constrained models may rely more heavily on explicit documentation, while larger models can infer intended behaviour from signatures alone.

The fact that five of seven models showed no significant effects despite testing on the same code snippets indicates that documentation alone is insufficient for most current LLMs. This contrasts with conventional software engineering wisdom emphasizing documentation importance (Boehm, 1984; Sommerville, 2011), suggesting that LLMs may process code differently than human developers.

Future research should investigate: (1) whether specific docstring formats (e.g., type hints vs. natural language descriptions) provide differential benefits; (2) whether docstrings interact with other contextual factors like code complexity; (3) why certain models (Codestral, Deepseek-V3) show sensitivity while others do not; and (4) whether docstring benefits emerge for more complex classes beyond our dataset’s scope.

RQ2 Findings: Comprehensive statistical analysis with appropriate multiple comparison corrections found significant improvements in only 2 of 7 models, and only for the full vs. partial comparison. Effect sizes were uniformly negligible.

6 Impact of Retrieval-Augmented Generation (RAG)

This section investigates whether retrieval-augmented generation (RAG) can further enhance functional correctness for unseen code by providing relevant examples from seen data. RQ3: *Can retrieval-augmented generation (RAG) using seen data improve LLM performance on unseen class-level code generation tasks?* We evaluate the same seven LLMs (Codestral, Deepseek-V3, GPT-4.1, GPT-5, GPT-OSS, Llama 4 Maverick, Qwen Coder 2.5) on the *RealClassEval* post-cutoff dataset with and without RAG-retrieved examples. The examples were retrieved from *RealClassEval* pre-cutoff data. The same performance metric of pass rate was used.

We emphasize a framing distinction before presenting results: RAG is an external retrieval-and-prompting strategy layered on top of a model, not a property of the model itself. The findings in this section therefore speak to the performance of a specific *retrieval-augmented pipeline* under our experimental conditions, and should not be conflated with claims about base LLM capability made in RQ1 and RQ2, which evaluate models without any retrieved context.

6.1 Approach

To assess RAG’s impact on unseen code generation, we employed a controlled experimental design comparing LLM performance with and without retrieved documentation from seen data.

6.1.1 Data Preparation

The evaluation utilized *RealClassEval*’s post-cutoff dataset, ensuring all test cases were from projects created after model training cutoffs. For each target class skeleton, RAG retrieved the top $k = 2$ most similar class skeletons from pre-cutoff data using cosine similarity on embeddings obtained from **GraphCodeBert** (Guo et al., 2020), then appended their complete implementations (signature, docstrings, and body) to the prompt.

We selected $k = 2$ to balance retrieval effectiveness with computational feasibility across our large-scale evaluation. This choice enables systematic investigation of RAG’s impact on class-level generation while maintaining practical scalability across seven models and 400 classes. Real-world class implementations vary significantly in length, and larger values of k would create prohibitively long prompts for many classes, potentially exceeding model context windows or incurring high computational costs that would limit experimental scope. Yang et al. (Yang et al., 2025) also frame the trade-off between retrieval-augmented performance gains and computational cost as a recognized, first-order design consideration in retrieval-augmented code generation pipelines in general. Our focus centers on demonstrating *whether* retrieval-augmented generation improves class-level code generation rather than optimizing retrieval hyperparameters, which is an important but distinct research question that would require dedicated parameter space exploration. The $k = 2$ configuration provides sufficient retrieved context to demonstrate RAG mechanisms (showing concrete implementation patterns and examples) while ensuring consistent evaluation conditions across all experimental settings. This choice is also consistent with prior findings in retrieval-augmented code generation. Chen et al. (Chen et al., 2026) report that in-context learning performance for repository-level code generation peaks with exactly two retrieved/demonstration examples across multiple models, with additional examples degrading performance due to context saturation. More broadly, retrieval-augmented approaches to code-related tasks do not consistently benefit from larger k : Meng et al. (Meng et al., 2025) find that top-1 retrieval outperforms larger retrieval

sets in code review generation, with additional retrieved items introducing redundancy and conflicting cues, and Wang et al. (Wang et al., 2023) use $k = 1$ in the retrieval-augmented generative setting for CodeT5+. Shi et al. (Shi et al., 2022) tested $k \in \{1, 3, 5, 7, 9\}$ for retrieval-augmented commit message generation and found performance was generally stable across this range, suggesting diminishing returns to exhaustive k -tuning once k is small. Together, this literature supports treating $k = 2$ as a reasonable, literature-consistent design choice rather than an arbitrary one. It does not, however, substitute for an ablation on our own data, which we discuss as a direction for future work given the computational cost constraints already noted above.

6.1.2 Study Design

Like RQ2, three docstring conditions were tested to investigate RAG’s interaction with documentation completeness: (1) **full docstrings**; (2) **partial docstrings**; and (3) **no docstrings**. Each condition was evaluated both with RAG (retrieved examples included) and without RAG, yielding paired samples for within-subjects analysis.

6.1.3 Statistical Analysis

No new statistical methodology has been used in RQ3 that has not been used in the previous two RQs. All the statistical tests applied in this study have been discussed in the previous section(s). Here is a quick summary of what steps we have taken to answer RQ3.

Given the paired structure (each snippet tested with and without RAG) and non-normal pass rate distributions, we again applied the Wilcoxon signed-rank test to compare pass rate differences (RAG - non-RAG) for each model-docstring combination. To control Type I errors while preserving statistical power, we applied the Benjamini-Hochberg False Discovery Rate (FDR) correction ($\alpha = 0.05$) *separately for each docstring condition* (7 model comparisons per condition). The decision to apply FDR correction separately per docstring type (rather than globally across all 21 tests) warrants justification. We treat the three docstring conditions as substantively distinct experimental contexts: full docstrings provide complete specifications (testing RAG’s value when information is abundant), partial docstrings create information gaps (testing RAG’s gap-filling capability), and no docstrings represent minimal guidance (testing RAG’s effectiveness without structural anchors). These conditions address fundamentally different research sub-questions about RAG’s mechanism, making separate FDR families appropriate. This choice was made because we want to understand RAG’s effect on different docstring types individually, and not collectively.

FDR-adjusted p-values are reported, with statistical significance defined as $p_{\text{FDR}} < 0.05$. Cliff’s Delta measured effect size (negligible < 0.147 , small < 0.33 , medium < 0.474 , large ≥ 0.474) with 95% bootstrap confidence intervals. For distributions with high skewness ($|skewness| > 1$), sign tests vali-

Table 7: Summary of RAG effects across docstring conditions on Post-Cutoff dataset. Positive mean differences indicate RAG outperforms non-RAG. Per-type FDR correction applied separately to each condition (7 tests per condition).

Docstring Condition	Mean Percentage Point Improvement	Improved	Worsened	Significant ($p_{FDR} 0.05$)
Full	1.55	81	51	0/7
Partial	4.25	125	55	5/7
No	3.58	141	77	0/7
Overall	3.11	347	183	5/21

dated Wilcoxon results. Power analysis for paired designs estimated the study’s ability to detect observed effect sizes.

6.2 Findings

RAG demonstrates significant benefits for code generation with partial docstrings, with 5 of 7 models showing statistically significant improvements after FDR correction. Benefits are negligible with full docstrings and marginal with no docstrings, revealing a critical interaction between RAG and documentation completeness. Table 7 summarizes results across docstring conditions, while Table 8 presents detailed model-specific findings organized by documentation level.

6.2.1 RAG with Full Docstrings: No Significant Benefits

When complete documentation is provided, RAG shows no significant improvements across all seven models (Table 8). Mean improvements range from -0.8% (GPT-5) to 3.1% (Deepseek-V3), with all $p_{FDR} > 0.21$. This pattern suggests that when models receive comprehensive docstrings, retrieved examples provide minimal additional value.

6.2.2 RAG with Partial Docstrings: Strong Significant Benefits

RAG demonstrates its strongest and most consistent benefits when documentation is incomplete, with 5 of 7 models achieving statistical significance after FDR correction (Table 8). Significant improvements range from 4.2% (Llama 4 Maverick, $p_{FDR}=0.024$) to 6.9% (Deepseek-V3, $p_{FDR}=0.023$), with adequate statistical power (0.634–0.838) supporting these findings. Codestral (5.3%, $p_{FDR}=0.030$), GPT-4.1 (4.6%, $p_{FDR}=0.023$), and Qwen Coder 2.5 (5.2%, $p_{FDR}=0.024$) also show significant benefits. Only GPT-5 shows negative effects (-0.6%, $p_{FDR}=0.739$), suggesting model-specific architectural differences in utilizing retrieved context.

Table 8: Model-specific RAG effects organized by docstring condition. A positive difference means RAG performed better, and vice versa. **Bold** entries indicate statistical significance.

Docstring Type	Model	Mean Pass Rate % Diff.	FDR-Adjusted p	Improved/Worsened
Full	Codestral	1.79	0.536	11/6
	Deepseek-V3	3.07	0.536	16/9
	GPT-4.1	0.84	0.617	6/4
	GPT-5	-0.77	0.798	12/13
	GPT-OSS	2.37	0.536	17/8
	Llama 4 Maverick	2.33	0.536	13/6
	Qwen Coder 2.5	0.94	0.617	6/5
Partial	Codestral	5.27	0.030	18/6
	Deepseek-V3	6.94	0.023	22/7
	GPT-4.1	4.63	0.023	16/5
	GPT-5	-0.58	0.739	15/18
	GPT-OSS	3.26	0.212	21/11
	Llama 4 Maverick	4.25	0.024	18/4
	Qwen Coder 2.5	5.18	0.024	15/4
No	Codestral	5.74	0.142	22/8
	Deepseek-V3	5.84	0.057	23/11
	GPT-4.1	2.75	0.357	18/13
	GPT-5	1.07	0.667	20/18
	GPT-OSS	2.20	0.488	18/12
	Llama 4 Maverick	2.15	0.357	18/8
	Qwen Coder 2.5	5.93	0.057	22/7

All Cliff’s Delta values, however, remain negligible (0.049–0.078), indicating modest practical effect sizes despite statistical significance. However, power analysis confirms adequate detection capability for the observed effects (power 0.634–0.838 for significant results), validating that these are genuine improvements rather than false positives.

6.2.3 RAG with No Docstrings: Marginal Non-Significant Benefits

Without documentation, RAG shows positive trends for most models but fails to achieve statistical significance after FDR correction. Deepseek-V3 (5.8%, $p_{\text{raw}}=0.016$, $p_{\text{FDR}}=0.057$) and Qwen Coder 2.5 (5.9%, $p_{\text{raw}}=0.014$, $p_{\text{FDR}}=0.057$) approach significance, while Codestral shows a strong trend (5.7%, $p_{\text{raw}}=0.061$, $p_{\text{FDR}}=0.142$). Power analysis reveals moderate detection capability (0.550–0.725), suggesting these effects may be genuine but require larger samples for definitive confirmation.

6.3 Interpretation

The findings reveal a critical interaction between RAG and documentation completeness, answering RQ3: the retrieval-augmented pipeline evaluated here

significantly improves code generation quality for unseen tasks, but only when documentation is incomplete. This “information gap hypothesis” (Loewenstein, 1994; Golman and Loewenstein, 2018) suggests RAG’s primary value lies in compensating for missing context rather than enhancing already-comprehensive specifications.

6.3.1 The Information Gap Hypothesis

The striking pattern—5/7 models significant with partial docstrings, 0/7 with full docstrings, 0/7 with no docstrings—supports a specific mechanistic interpretation. When documentation is complete, models already possess sufficient behavioural specifications; retrieved examples offer redundant information. When documentation is absent, models lack structured guidance to effectively leverage retrieved examples—the context is too noisy without docstrings to anchor understanding. The “sweet spot” occurs with partial docstrings: enough structure to interpret retrieved examples, but insufficient detail to implement correctly without them.

6.3.2 Practical Significance Despite Negligible Effect Sizes

Although all Cliff’s Delta values are negligible (< 0.147), the improvements have practical importance. A 4–7% pass rate increase translates to 0.12–0.28 additional test cases passed per class with a very restrictive assumption of mean test counts of 3–4 per class. For large codebases generating thousands of classes, this compounds to substantial quality gains. Moreover, combining RQ2’s docstring benefits (mean 1.25%) with RQ3’s RAG benefits (4.25% for partial docstrings) yields a potential 5.5% cumulative improvement—equivalent to approximately 0.19 tests per class, or roughly 190 additional correct implementations per 1,000 generated classes. Additionally, RQ2 showed benefits of up to 2.7% with full docstrings, while RQ3 shows 4.3–6.9% with partial docstrings plus RAG. This suggests RAG and documentation are complementary strategies, with RAG providing greater relative benefits when documentation is lacking.

6.3.3 Model-Specific Patterns and GPT-5’s Anomaly

Five models (Codestral, Deepseek-V3, GPT-4.1, Llama 4 Maverick, Qwen Coder 2.5) show consistent positive responses to RAG with partial docstrings, while GPT-5 exhibits negative effects (−0.8% full, −0.6% partial). This suggests architectural differences in context utilization. GPT-5 may suffer from “context interference” (retrieved examples conflicting with its training-based priors) or may weight retrieved context differently than documentation. GPT-OSS shows positive trends (3.3% partial) but insufficient power for significance, suggesting moderate responsiveness.

The consistency of the pattern across five diverse architectures (transformer variants, different training datasets, varying parameter counts) strengthens

the generalizability of the “information gap” hypothesis. RAG’s effectiveness is not model-specific but rather a property of the interaction between retrieval quality and documentation completeness. However, we do acknowledge that further investigation is required to conclusively state why these inter-model differences are being observed.

6.3.4 Interpret with Caution

It is worth mentioning that power analysis revealed that this study’s power varies substantially across conditions (0.059–0.838), with full and no docstring conditions showing inadequate power (mostly < 0.30) to detect true effects. This explains why full docstrings show no significant results despite positive trends, meaning that the study cannot reliably distinguish small RAG benefits from noise. However, the partial docstring condition achieves adequate power (0.634–0.838) for significant results, validating these findings. Future work should increase sample sizes to achieve higher power for detecting true effects across all conditions.

RQ3 Findings: This study provides systematic evidence that the effectiveness of the retrieval pipeline evaluated here depends critically on documentation completeness, with the strongest benefits occurring when models receive partial documentation. This finding bridges RQ2’s insights on documentation effects and suggests a path toward more efficient code generation, though it speaks to this specific retrieval configuration rather than retrieval augmentation in general.

7 Error Analysis

To understand the nature of failures in LLM-generated code, this section addresses RQ4: *What are the most common errors in LLM-generated class-level code?* We analyze error distributions across all experimental conditions from RQ1–RQ3, examining *all failed test cases* across seven LLMs, three datasets, and multiple documentation and retrieval configurations. We characterize overall error patterns, compare synthetic versus real-world failure modes, and investigate how docstrings and RAG affect error types.

7.1 Approach

To comprehensively characterize error patterns in LLM-generated code, we employed a multi-level statistical analysis examining error type distributions across experimental conditions.

7.1.1 Error Classification and Data Processing

We extracted error types from test execution reports across all conditions, parsing exception types from failure messages. Errors were classified into Python’s standard exception taxonomy: `AttributeError`, `TypeError`, `AssertionError`, `ValueError`, `NameError`, `KeyError`, `IndexError`, `ZeroDivisionError`, `FileNotFoundError`, `ImportError`, `RuntimeError`, and `SyntaxError`. Non-standard errors, such as API specific exceptions, were aggregated into an “Other” category. For each failed class, we tabulated error counts by type, aggregating across test cases to create error frequency distributions.

7.1.2 Statistical Analysis

We applied chi-square tests of independence (Pearson, 1900; Fisher, 1970) to compare error type distributions across conditions, appropriate for categorical error frequency data. The analysis comprised four complementary comparisons:

Overall Distribution Analysis: Chi-square goodness-of-fit test (Conover, 1999) against uniform distribution was applied to confirm non-uniform error patterns in order to establish that error types are non-randomly distributed across failures. The larger the chi-squared value, the greater the deviation from the uniform distribution, implying that there is a meaningful pattern worth investigating.

In our context, this test can be realized as follows: Let p_i denote the probability of observing error type i where $i \in \{\text{AttributeError}, \text{TypeError}, \dots, \text{Other}\}$. We tested:

$$\begin{aligned} H_0 : p_i &= \frac{1}{13} \quad \forall i \in \{1, 2, \dots, 13\} \\ H_A : \exists i \text{ such that } p_i &\neq \frac{1}{13} \end{aligned} \tag{4}$$

with test statistic $\chi^2 = \sum_{i=1}^{13} \frac{(O_i - E_i)^2}{E_i}$ where O_i is the observed count and $E_i = \frac{\sum_{i=1}^{13} O_i}{13}$ is the expected count under uniform distribution. The test has $df = 12$ degrees of freedom.

Like previous RQs, Cramér’s V for effect size and bootstrap 95% confidence intervals (CIs) to quantify heterogeneity were also applied.

Dataset-Wise Comparison: Pairwise chi-square tests were applied to compare error distributions across ClassEval, pre-cutoff, and post-cutoff datasets for each model (3 comparisons $\times$ 7 models = 21 tests). Global Benjamini-Hochberg FDR correction ($\alpha = 0.05$) controlled Type I error across all tests, consistent with RQ1 methodology. Cramér’s V quantified effect sizes with thresholds: negligible (< 0.1), small (< 0.3), medium (< 0.5), large (≥ 0.5).

Docstring Impact: Chi-square tests were again applied to compare error distributions for full vs. partial and full vs. no docstrings within pre-cutoff

and post-cutoff datasets separately (2 comparisons per model-dataset pair). Per-dataset FDR correction (14 tests per dataset) followed RQ2’s approach of treating temporal contexts as distinct experimental families.

RAG Impact: Another set of Chi-square tests was applied to compare RAG vs. non-RAG error distributions within each docstring condition for post-cutoff data (7 models per condition). Per-docstring-condition FDR correction (7 tests per condition) mirrored RQ3’s methodology, treating documentation levels as substantively different information environments.

For all comparisons, we computed statistical power to detect observed effect sizes, ensuring reliable inference. Bootstrap CIs (1,000 iterations) provided robust uncertainty estimates for effect sizes. To contextualize statistical findings, we computed percentage point differences in error type frequencies, identifying which errors increase or decrease across conditions.

7.2 Findings

The analysis revealed three major patterns: (1) a consistent error signature dominated by **AttributeError**, **TypeError**, and **AssertionError**; (2) fundamental differences in error types between synthetic benchmarks and real-world code; and (3) RAG-induced error substitution where retrieval prevents certain failures while introducing others. Table 9 summarizes the overall distribution, Tables 10 and 11 detail dataset comparisons, and Table 12 characterizes RAG’s error substitution mechanism.

7.2.1 Overall Error Distribution

Across all failed test cases, three error types dominate LLM-generated code failures (Table 9). **AttributeError** accounts for 43.84% of all errors, **TypeError** for 21.65%, and **AssertionError** for 18.51%. These top three error types collectively represent 84% of all failures, indicating a concentrated failure signature. Chi-square goodness-of-fit testing against a uniform distribution confirmed highly non-uniform patterns ($\chi^2 = 37847.3$, $p < 10^{-300}$), with “Other” errors contributing 10.42%. Notably, **SyntaxError** accounted for 0% of failures, demonstrating that modern LLMs generate syntactically valid Python code consistently across all experimental conditions.

7.2.2 Dataset-Wise Comparison: Synthetic vs. Real-World Error Patterns

Error distributions differ fundamentally between synthetic benchmarks and real-world projects (Figure 4), with large effect sizes across all model-dataset comparisons (Table 10). All 21 pairwise comparisons showed statistically significant differences after FDR correction. ClassEval vs. pre-cutoff and ClassEval vs. post-cutoff comparisons yielded large effect sizes (Cramér’s V: 0.59–0.85, all $p_{\text{FDR}} < 10^{-116}$), while pre-cutoff vs. post-cutoff comparisons showed smaller but still significant differences (Cramér’s V: 0.22–0.28, all $p_{\text{FDR}} <$

Table 9: Overall error type distribution across all conditions.

Error Type	Count	Percentage (%)
AttributeError	11,270	43.84
TypeError	5,566	21.65
AssertionError	4,759	18.51
Other	2,679	10.42
ValueError	702	2.73
NameError	339	1.32
KeyError	156	0.61
FileNotFoundError	89	0.35
RuntimeError	46	0.18
ZeroDivisionError	39	0.15
IndexError	38	0.15
ImportError	22	0.09
SyntaxError	0	0.00

Table 10: Dataset-wise error distribution comparison with global FDR correction (21 tests). All comparisons are significant after correction.

Model	Comparison	p_{raw}	p_{FDR}	Effect Size
Codestral	ClassEval vs Pre-Cutoff	5.46e-144	1.04e-143	Large
	ClassEval vs Post-Cutoff	3.10e-116	4.65e-116	Large
	Pre-Cutoff vs Post-Cutoff	1.01e-28	1.32e-28	Small
Deepseek-V3	ClassEval vs Pre-Cutoff	5.91e-164	2.07e-163	Large
	ClassEval vs Post-Cutoff	1.75e-142	3.07e-142	Large
	Pre-Cutoff vs Post-Cutoff	3.23e-30	4.52e-30	Small
GPT-4.1	ClassEval vs Pre-Cutoff	1.01e-164	4.23e-164	Large
	ClassEval vs Post-Cutoff	1.43e-136	2.32e-136	Large
	Pre-Cutoff vs Post-Cutoff	3.89e-24	4.80e-24	Small
GPT-5	ClassEval vs Pre-Cutoff	2.17e-176	2.72e-175	Large
	ClassEval vs Post-Cutoff	2.52e-157	7.56e-157	Large
	Pre-Cutoff vs Post-Cutoff	5.45e-24	6.35e-24	Small
GPT-OSS	ClassEval vs Pre-Cutoff	1.76e-168	9.25e-168	Large
	ClassEval vs Post-Cutoff	4.94e-145	1.04e-144	Large
	Pre-Cutoff vs Post-Cutoff	3.77e-18	3.77e-18	Small
Llama 4 Maverick	ClassEval vs Pre-Cutoff	2.59e-176	2.72e-175	Large
	ClassEval vs Post-Cutoff	4.71e-148	1.10e-147	Large
	Pre-Cutoff vs Post-Cutoff	4.33e-21	4.79e-21	Small
Qwen Coder 2.5	ClassEval vs Pre-Cutoff	6.69e-176	4.68e-175	Large
	ClassEval vs Post-Cutoff	3.04e-150	7.97e-150	Large
	Pre-Cutoff vs Post-Cutoff	1.36e-19	1.42e-19	Small

10^{-18}). Statistical power exceeded 0.99 for all tests, ensuring reliable detection of effects.

Detailed analysis of error type frequencies by dataset (Table 11) reveals the source of these differences. ClassEval’s error profile is dominated by **AssertionError** (71.80%), reflecting the logic-focused nature of synthetic test suites, with **NameError** (14.44%) as the second most common failure. On the other hand, real-world datasets show **AttributeError** dominance (49.41% pre-cutoff, 45.67% post-cutoff) and substantial **TypeError** frequencies (22.14%



Fig. 4: Distribution of Error Types Across Datasets.

Table 11: Error type distributions by dataset (%). ClassEval shows AssertionError dominance, while real-world datasets show AttributeError and TypeError prominence.

Error Type	ClassEval	Pre-Cutoff	Post-Cutoff
AssertionError	71.80	5.34	17.57
AttributeError	2.14	49.41	45.67
TypeError	2.33	22.14	23.78
NameError	14.44	0.12	0.27
Other	0.63	18.29	9.16
ValueError	5.30	2.81	2.53
ZeroDivisionError	1.51	0.14	0.00
KeyError	1.07	0.27	0.61
IndexError	0.44	0.42	0.01
FileNotFoundError	0.29	0.76	0.18
RuntimeError	0.05	0.30	0.20
ImportError	0.00	0.00	0.03
SyntaxError	0.00	0.00	0.00

pre-cutoff, 23.78% post-cutoff), while **AssertionError** drops to 5.34% (pre-cutoff) and 17.57% (post-cutoff).

Quantifying these differences reveals striking patterns. ClassEval produces 66.47 percentage points (pp) more **AssertionError** than pre-cutoff and 54.24 pp more than post-cutoff, while showing 47.27 pp fewer **AttributeError** than pre-cutoff and 43.53 pp fewer than post-cutoff. **NameError** is 14.32 pp more common in ClassEval than pre-cutoff and 14.17 pp more than post-cutoff. Conversely, real-world code exhibits 19.80–21.45 pp more **TypeError** than ClassEval. These differences directly explain RQ1’s performance gap: synthetic benchmarks test logical correctness through assertions, while real-world projects require correct object attribute access and type consistency—capabilities LLMs struggle to master.

The small but significant differences between pre-cutoff and post-cutoff (Cramér’s V: 0.22–0.28) contrast with the negligible pass rate differences found

in RQ1. While seen and unseen real-world code exhibit similar overall correctness, they show subtle error pattern variations. pre-cutoff code produces more “Other” errors (18.29% vs. 9.16%), suggesting idiosyncratic project-specific failures in older codebases, while post-cutoff shows slightly more **AssertionError** (17.57% vs. 5.34%), potentially reflecting evolution in testing practices toward more explicit assertions in recent projects.

7.2.3 Docstring Impact on Error Patterns

Similar to the docstrings’ effect on overall pass rate (RQ2), they do not significantly affect error type distributions either. Of 28 pairwise comparisons (full vs. partial, full vs. no, across 7 models and 2 datasets), none achieved statistical significance after FDR correction (all $p_{\text{FDR}} > 0.05$). Effect sizes ranged from negligible to small (Cramér’s V: 0.04–0.18), with most comparisons showing negligible effects. Power analysis revealed, on average, moderate detection capability (power: 0.19–0.92), suggesting that observed non-significance reflects a genuine absence of large effects.

This finding indicates that docstrings do not change error types even if, in isolated cases, they improve the pass rate. Models make the same categories of mistakes (primarily **AttributeError**, **TypeError**, and **AssertionError**) regardless of documentation completeness. Documentations affect all error types proportionally, rather than preventing or increasing specific failure modes selectively.

7.2.4 RAG Error Substitution Mechanism

An overall pattern of RAG’s effect on error distributions is shown in Figure 5. RAG not only improves pass rates with partial docstrings (RQ3) but also significantly alters error type distributions for three models: Deepseek-V3 ($\chi^2 = 20.68$, $p_{\text{FDR}} = 0.030$), GPT-5 ($\chi^2 = 16.62$, $p_{\text{FDR}} = 0.047$), and Llama 4 Maverick ($\chi^2 = 18.78$, $p_{\text{FDR}} = 0.047$), all with partial docstrings. Effect sizes are small (Cramér’s V: 0.14–0.16) but statistically significant with adequate power (0.86–0.94). No significant differences emerged for full or no docstring conditions, consistent with RQ3’s “information gap hypothesis.”

Detailed analysis of error substitution patterns (Table 12) reveals RAG’s dual mechanism: substantial reductions in logic and object access errors, coupled with occasional increases in dependency-related failures. Aggregating across the three significant models, RAG with partial docstrings reduces 13.31% of all errors, but the reduction is not uniform across error types.

AttributeError shows the largest absolute reduction (639 $\rightarrow$ 579, –60 instances), followed by “Other” errors (129 $\rightarrow$ 76, –53), **AssertionError** (233 $\rightarrow$ 190, –43), **ValueError** (46 $\rightarrow$ 28, –18), and **TypeError** (320 $\rightarrow$ 304, –16). These reductions suggest that retrieved examples help models understand correct object attribute access patterns, type specifications, and logical requirements, preventing the most common failure modes.

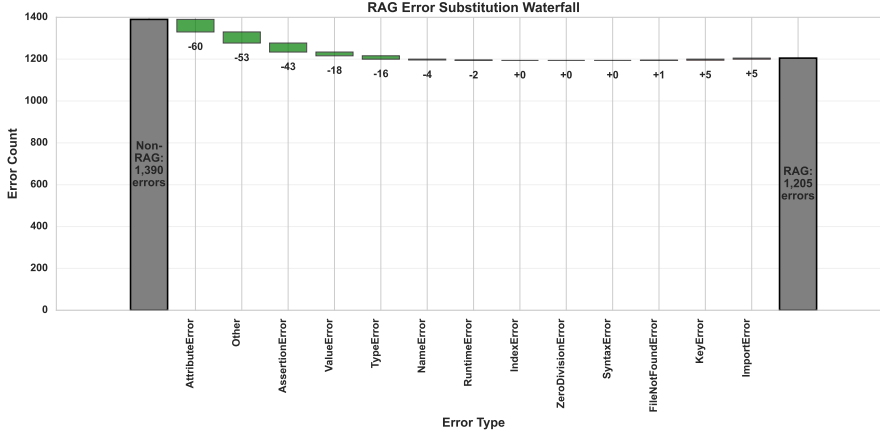


Fig. 5: Error Type Changes with RAG Implementation.

However, RAG introduces new failures in specific categories. **ImportError** increases from 2 to 7 instances (+5, +250%), and **KeyError** from 11 to 16 (+5, +45%). These increases reflect a substitution mechanism: retrieved examples contain dependencies or data structures absent in the target class, leading models to generate code that references unavailable imports or dictionary keys. The pattern is particularly pronounced in GPT-5, which shows a substantial increase in **ImportError** (0 → 7), and Deepseek-V3, which also exhibits a significant increase in **KeyError** (1 → 12).

Interestingly, while **AttributeError** and **TypeError** decrease in absolute count, their relative proportions increase (45.97% → 48.05% and 23.02% → 25.23%, respectively). This occurs because other error types decrease more substantially, making these harder-to-eliminate errors more prominent in the residual failure set. This suggests that **AttributeError** and **TypeError** represent fundamental challenges in class-level code generation that retrieval cannot fully address, as they require deep semantic understanding of object hierarchies and type systems that transcend pattern matching.

7.3 Interpretation

The error analysis reveals three critical insights into LLM class-level code generation failures: a concentrated error signature dominated by object access and type consistency issues, fundamental differences between synthetic and real-world failure modes, and a RAG-induced error substitution mechanism. These findings provide actionable guidance for both practitioners and researchers seeking to improve code generation quality.

Table 12: RAG error substitution patterns for significant models with partial docstrings (aggregate across Deepseek-V3, GPT-5, Llama 4 Maverick). Negative differences indicate reductions, positive differences indicate increases.

Error Type	Non-RAG Count	RAG Count	Absolute Difference	Non-RAG %	RAG %	Percentage Point Difference
<i>Reductions</i>						
AttributeError	639	579	-60	45.97	48.05	+2.08
Other	129	76	-53	9.28	6.31	-2.97
AssertionError	233	190	-43	16.76	15.77	-0.99
ValueError	46	28	-18	3.31	2.32	-0.99
TypeError	320	304	-16	23.02	25.23	+2.21
NameError	6	2	-4	0.43	0.17	-0.26
<i>Increases</i>						
ImportError	2	7	+5	0.14	0.58	+0.44
KeyError	11	16	+5	0.79	1.33	+0.54
FileNotFoundError	2	3	+1	0.14	0.25	+0.11

7.3.1 The *AttributeError-TypeError-AssertionError* Triad

The dominance of three error types, namely **AttributeError** (43.84%), **TypeError** (21.65%), and **AssertionError** (18.51%), accounting for 84% of all failures, reveals a consistent failure signature across models and conditions. This concentration indicates that LLM code generation struggles stem from specific, identifiable challenges rather than diffuse implementation errors.

AttributeError predominance directly links to RQ1’s qualitative finding that real-world test suites involve complex type metadata checks and object hierarchies. LLMs generate plausible class structures but fail to correctly implement attribute access patterns, particularly in inheritance hierarchies and dynamic attribute resolution. This suggests that current training paradigms, which excel at surface-level syntax and common patterns, do not sufficiently capture deep object-oriented semantics.

TypeError frequency reflects challenges in maintaining type consistency across method calls, particularly in dynamically typed Python, where type annotations may be incomplete or absent. RQ2 showed that docstrings with explicit type information improve correctness by 1–3%, but the persistence of **TypeError** even with full documentation suggests that models struggle to propagate type constraints through code generation, often mixing incompatible types or calling methods with incorrect argument types.

AssertionError prevalence indicates logical implementation failures where generated code produces incorrect values despite syntactic validity. The stark difference between ClassEval (71.80%) and real-world datasets (5.34–17.57%) demonstrates that synthetic benchmarks primarily test logical correctness through explicit assertions, while real-world failures manifest through object access and type issues before reaching assertion points.

The absence of **SyntaxError** (0.00%) confirms that modern LLMs have mastered Python syntax completely. Even though this is opposite to the bug patterns in function-level LLM-generated code reported in a prior study (Tambon et al., 2025), our finding validates LLMs’ use for code generation tasks

and shifts attention to semantic correctness—understanding object models, type systems, and logical specifications—as the remaining frontier.

7.3.2 Synthetic vs. Real-World Error Type Divergence

The large effect sizes (Cramér’s V: 0.59–0.85) comparing ClassEval to real-world datasets definitively answer RQ1’s open question about *why* performance gaps exist. Synthetic benchmarks do not merely present easier versions of the same challenges; they represent fundamentally different problem spaces with distinct failure modes.

ClassEval’s assertion-dominated error profile (71.80% **AssertionError**) reflects its design philosophy: test logical correctness through input-output matching with simple equality checks. This aligns with the qualitative analysis in RQ1, which found ClassEval test suites use straightforward assertions like numerical comparisons and dictionary checks, requiring no external dependencies. LLMs perform well (83.67–88.51% pass rate) because they can generate logically correct implementations for well-specified pure functions.

Real-world code’s attribute-and-type-dominated profile (45–49% **AttributeError**, 22–24% **TypeError**) reflects genuine software engineering challenges: navigating complex object hierarchies, managing dependencies, and maintaining type consistency across diverse contexts. RQ1’s qualitative analysis revealed that real-world test suites involve system dependencies (e.g., **numpy**, **MCPContext**) and metadata checks, leading to **AttributeError** when models misunderstand object structures. The 47 percentage point increase in **AttributeError** from ClassEval to pre-cutoff quantifies this fundamental shift.

The **NameError** pattern (14.44% ClassEval vs. 0.12–0.27% real-world) reveals another dimension of divergence. In ClassEval, models often fail to correctly reference variables or function names in self-contained implementations, while real-world code’s lower **NameError** rate suggests that context from surrounding project code helps models resolve names correctly. This paradoxically indicates that isolated synthetic tasks may be harder for name resolution than contextualized real-world scenarios.

The small but significant pre-cutoff versus post-cutoff differences (Cramér’s V: 0.22–0.28) warrant brief discussion despite their minor magnitude. While RQ1 found negligible pass rate differences between seen and unseen real-world code, error patterns show subtle temporal evolution. pre-cutoff code produces more “Other” errors (18.29% vs. 9.16%), suggesting older projects contain more idiosyncratic failure modes or deprecated patterns unfamiliar to models. post-cutoff’s higher **AssertionError** (17.57% vs. 5.34%) may reflect modern testing practices favouring explicit assertions, though this requires further investigation.

7.3.3 RAG Error Substitution: Trading Logic Errors for Dependency Failures

RQ3 demonstrated that RAG improves pass rates by 4–7% with partial docstrings, but the error distribution analysis reveals the *mechanism*: RAG pre-

vents logic and object access failures while introducing dependency-related errors. This substitution pattern is not merely error reduction but error transformation.

The substantial reductions in `AttributeError`, `AssertionError`, and `ValueError` indicate that retrieved examples provide concrete implementations that models can adapt, particularly for object attribute access patterns and logical requirements. When partial docstrings specify *what* methods should do but not *how*, retrieved similar implementations fill the gap, showing models correct attribute usage and logic structure.

However, the increases in `ImportError` and `KeyError` reveal RAG’s limitation: retrieved examples may contain dependencies or data structures absent in the target context. Models blindly copy import statements or dictionary key accesses from examples without verifying availability in the target class. GPT-5’s dramatic `ImportError` increase ($0 \rightarrow 7$) suggests that this model particularly struggles to filter context-specific dependencies from retrieved examples, while Deepseek-V3’s `KeyError` explosion ($1 \rightarrow 12$) indicates difficulty distinguishing data structure patterns from concrete key names.

The relative proportion increases for `AttributeError` (+2.08 pp) and `TypeError` (+2.21 pp) despite absolute reductions, revealing these as “hard-core” errors resistant to retrieval-based mitigation. Even with concrete examples, models struggle to fully master object attribute navigation and type consistency, suggesting these require deeper semantic understanding beyond pattern matching.

It is worth mentioning that Llama 4 Maverick shows the most balanced substitution profile, reducing errors across the board with minimal dependency increases, suggesting architectural differences in how models integrate retrieved context. Future work should investigate these model-specific retrieval utilization strategies to optimize RAG effectiveness.

7.3.4 Reflection on Previous RQs

RQ4 cohesively integrates with prior research questions. RQ1 identified a performance gap between synthetic and real-world code (83.60–88.27% vs. 24.59–33.60% pass rates); RQ4 explains this through fundamentally different error types: `ClassEval` tests logical correctness through assertions, while real-world code requires object attribute mastery. RQ2 showed docstrings improve correctness by 1–3% even though mostly statistically non-significant; RQ4 reveals they reduce all error types proportionally without changing failure modes, clarifying the documentation’s mechanism. RQ3 demonstrated RAG’s 4–7% improvement with partial docs; RQ4 exposes the error substitution mechanism, trading logic failures for dependency errors, explaining both benefits and limitations.

RQ4 Findings: LLM class-level code generation failures follow a consistent, concentrated signature dominated by object access and type errors. Synthetic benchmarks fundamentally mischaracterize these challenges by focusing on logical assertions. RAG offers promising but partial mitigation through error substitution.

8 Discussion

This study provides the first comprehensive evaluation of LLM performance on class-level code generation using real-world projects, revealing fundamental gaps between synthetic benchmark performance and practical capabilities. In this section, we provide a combined summary of all RQs and share actionable insights for practitioners and researchers.

8.1 Key Findings

8.1.1 Performance Gap Between Synthetic and Real-World Code

Our investigation of LLM performance across synthetic and real-world datasets revealed a sharp divide that questions current evaluation practices. All seven models achieved high pass rates on ClassEval, matching previous research findings. However, their performance dropped dramatically on real-world projects. The very small differences between pre-cutoff and post-cutoff show that models handle both seen and unseen real-world code equally well—but in both cases, the performance is poor for class-level code generation.

Our detailed analysis showed that this gap comes from different task types rather than harder problems. ClassEval test suites use simple equality checks with few external requirements, allowing models to achieve high pass rates through basic logic. Real-world test suites involve complex type checks, external system requirements, and integration between components—patterns consistent with genuine object-oriented implementation difficulty, though part of this gap may also reflect the absence of repository-level context.

8.1.2 Docstrings as Incremental Improvement

Our systematic test across three documentation levels (full, partial, no docstrings) revealed that complete docstrings may provide improvements in isolated cases. The pattern of benefits differed between time periods, with stronger effects on pre-cutoff data compared to post-cutoff. This suggests models rely more on documentation for familiar code patterns, where specifications help clarify how to implement things. However, comparing across time periods showed large differences between individual models. Some models performed better with post-cutoff data, while others performed better with pre-cutoff

data. This indicates that docstring effectiveness depends on how models are built and what data they were trained on, rather than simply whether the code was seen during training.

8.1.3 RAG’s Information Gap Hypothesis

Our retrieval pipeline showed a clear pattern with documentation completeness, supporting an “information gap hypothesis” for this configuration: retrieval helps most when documentation is incomplete. This pattern shows how RAG works: when documentation explains what the code should do but doesn’t show how to implement it (partial docstrings), retrieved examples fill this gap by providing actual code implementations.

Notably, GPT-5 performed worse with RAG, suggesting differences in how this model processes context. However, five other models with different designs (Codestral, Deepseek-V3, GPT-4.1, Llama 4 Maverick, Qwen Coder 2.5) all showed consistent positive results. This strengthens the applicability of the “information gap” hypothesis across models, for the retrieval configuration studied here.

8.1.4 Error Signatures and Substitution Mechanisms

Error analysis revealed a focused pattern of failures: `AttributeError`, `TypeError`, and `AssertionError` make up 84% of all failures. Notably, `SyntaxError` contributed 0%, confirming that modern LLMs have fully learned Python syntax, making semantic correctness (understanding meaning) the main remaining challenge.

Dataset comparison revealed very different types of errors between synthetic and real-world code. ClassEval’s error profile is mostly `AssertionError`, reflecting tests that check logic, while real-world datasets show mostly `AttributeError` and `TypeError`, reflecting real complexity in object-oriented programming.

RAG’s impact on error distributions revealed a pattern of replacing one error with another rather than reducing all errors equally. Across the three models with strong RAG effects (Deepseek-V3, GPT-5, Llama 4 Maverick), RAG reduced `AttributeError`, `AssertionError`, and `ValueError`, showing that retrieved examples help prevent logic mistakes and problems accessing objects. However, RAG increased `ImportError` and `KeyError`, suggesting that retrieved examples contain dependencies or data structures not present in the target code, which models copy without checking if they fit.

8.2 Class-Level Generation: The Critical Research Gap

Class-level code generation occupies a crucial but understudied position in the granularity spectrum between well-researched function-level and rapidly advancing repository-level generation. Rather than representing an arbitrary

choice of scope, class-level evaluation addresses a fundamental research gap that limits our understanding of LLM capabilities and constrains the effectiveness of repository-level approaches.

8.2.1 The Granularity Progression Problem

Current code generation research exhibits a striking discontinuity. Function-level generation has been extensively studied through benchmarks like HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), and their variants, establishing a clear understanding of LLM capabilities for isolated, single-purpose code units. Repository-level generation has emerged as the frontier challenge, with benchmarks like SWE-bench (Jimenez et al., 2023), HumanEvo (Zheng et al., 2024), and CodeAgent (Zhang et al., 2024) addressing complex multi-file scenarios.

However, this progression skips a critical intermediate step: *Can LLMs reliably implement individual classes given clear specifications?* Repository-level tasks inherently require successful class implementation—SWE-bench issues often involve creating or modifying individual classes, CodeAgent tools ultimately generate class implementations, and environment-aware generation must ensure classes execute correctly across configurations. Yet existing evaluations cannot isolate whether failures stem from fundamental class-level implementation deficiencies or repository-level context management challenges.

8.2.2 The Diagnostic Necessity

Class-level evaluation serves a crucial diagnostic function that repository-level benchmarks cannot provide. When a repository-level approach fails, the failure could originate from multiple sources: inadequate context retrieval, poor dependency resolution, insufficient architectural scaffolding, or simply the inability to implement individual classes correctly. Without understanding baseline class-level capabilities, researchers cannot determine where to focus their architectural innovations.

For instance, if an LLM cannot correctly implement a class given a complete specification and relevant imports, then sophisticated repository-level context management becomes irrelevant—the fundamental implementation capability is missing. Conversely, if class-level implementation succeeds but repository-level tasks fail, this localizes the problem to context management and dependency resolution, guiding research toward more effective retrieval and architectural strategies.

8.2.3 The Prerequisites for Repository-Level Success

Repository-level generation approaches make implicit assumptions about class-level capabilities that have never been systematically validated. Consider the typical repository-level workflow:

1. **Context Retrieval:** Gathering relevant code snippets, documentation, and dependency information
2. **Dependency Resolution:** Understanding cross-file relationships and import requirements
3. **Code Implementation:** Actually generating the required class or function code
4. **Integration:** Ensuring generated code integrates correctly with the existing repository structure

Current repository-level work focuses primarily on steps 1, 2, and 4—sophisticated retrieval mechanisms, dependency analysis, and integration strategies. However, step 3 receives minimal attention, with the implicit assumption that LLMs can successfully implement individual classes once provided with appropriate context.

Our findings challenge this assumption. If LLMs achieve only 25–34% correctness on isolated class implementation (as our results demonstrate), then even perfect context management cannot overcome this fundamental limitation. Repository-level approaches may be optimizing the wrong problem—developing increasingly sophisticated context management for models that cannot reliably implement the basic building blocks.

8.2.4 Isolating Core Capabilities from Architectural Scaffolding

Class-level evaluation enables precise measurement of core LLM implementation capabilities independent of tool augmentation, architectural scaffolding, or sophisticated prompt engineering. This isolation serves several critical research purposes:

Capability Assessment: Determining the baseline implementation abilities that repository-level approaches can build upon, establishing realistic expectations for what context management and retrieval can achieve.

Error Localization: Identifying whether failures originate from fundamental implementation deficiencies (attribute resolution, type consistency, object-oriented reasoning) or context management issues, enabling targeted research efforts.

Progress Measurement: Providing a stable evaluation target that tracks improvements in core generation capabilities rather than confounding implementation ability with architectural sophistication.

Transfer Understanding: Establishing whether implementation patterns learned from repository context transfer to novel classes, or whether each class requires extensive contextual scaffolding.

8.2.5 The Object-Oriented Foundation

Classes represent the fundamental abstraction in object-oriented programming, encapsulating state, behaviour, and relationships in ways that individual functions cannot capture. Class-level generation requires understanding:

- **State Management:** Maintaining object state across method calls and lifecycle events
- **Method Interaction:** Ensuring methods work cohesively to implement class contracts
- **Inheritance Hierarchies:** Correctly implementing parent-child relationships and method overriding
- **Interface Compliance:** Satisfying abstract method requirements and protocol adherence
- **Encapsulation Principles:** Proper access control and information hiding

These capabilities form the foundation for larger software systems. A repository consists of interacting classes; understanding how well LLMs can implement individual classes is a prerequisite to understanding how they can compose multiple classes into coherent systems.

In summary, class-level code generation represents neither an arbitrary granularity choice nor competition with repository-level work. Instead, it addresses a critical research gap that constrains understanding of LLM capabilities and limits the effectiveness of more sophisticated approaches. By systematically evaluating class-level implementation independent of repository context, we provide the diagnostic foundation necessary for advancing the entire spectrum of code generation research.

8.3 Broader Implications

This study provides the first comprehensive evaluation of LLM performance on class-level code generation using real-world projects, revealing fundamental gaps between synthetic benchmark performance and practical capabilities. In this section, we discuss actionable insights for practitioners, researchers, and the broader software engineering community.

8.3.1 The Simplicity-Complexity Paradox in Class Generation

Our convergence with RepoClassBench (Deshpande et al., 2024) reveals a striking paradox that fundamentally challenges current assumptions about class-level code generation approaches. Despite vastly different methodological complexity (*i.e.*, our class skeletons versus their natural language descriptions with repository context), both approaches achieve nearly identical baseline performance (25–34% vs. 24–34%). This convergence exposes a critical insight: **specification richness and contextual sophistication do not improve fundamental class implementation capability**. Our findings demonstrate that simple structural specifications (class signatures and method skeletons) achieve the same baseline effectiveness as complex natural language descriptions combined with repository context. This convergence is suggestive rather than conclusive, given the two studies’ non-comparable evaluation instruments and task setups. With that caveat, implementation-level factors such as attribute handling, type consistency, and object-oriented logic may represent a

meaningful bottleneck alongside specification richness or repository context, rather than being fully resolved by either. LLMs understand *what* to generate regardless of specification complexity; the challenge lies in generating *correct implementations*. Furthermore, RepoClassBench’s 40–54 percentage point improvements through RRR’s extensive tool scaffolding demonstrate what becomes possible when core limitations are circumvented through architectural solutions. However, their need for iterative tool-based refinement, static analysis integration, and agent-based reasoning frameworks underscores that these improvements come through *working around* rather than *solving* the fundamental implementation bottleneck we identify. While their approach successfully enables repository-level class generation, it requires substantial computational overhead and system complexity.

For practitioners, this convergence suggests that effective class generation tools can be built using simpler approaches that focus on the core implementation problem rather than elaborate specification mechanisms. The path to improved performance lies in addressing implementation-specific weaknesses, such as attribute handling, type consistency, and object-oriented semantics, rather than enhancing specification richness or contextual sophistication.

8.3.2 Rethinking Benchmark Design and Evaluation

The large performance gap between synthetic and real-world code (57–62 percentage points) and fundamentally different error patterns reveal critical limitations in current evaluation practices. If synthetic benchmarks primarily test logical correctness through simple assertions while real-world code requires an understanding of object-oriented concepts, then benchmark leaderboards provide misleading assessments of practical capabilities.

Future benchmarks must preserve realistic object structures, dependencies, and testing practices rather than simplifying tasks to basic function-level problems. The complete absence of `SyntaxError` across all conditions demonstrates that correct syntax generation is solved, and research focus should shift entirely to semantic understanding and correctness. This necessitates evaluation metrics that measure understanding of meaning: correct object attribute usage, type consistency, and dependency management.

8.3.3 Practical Deployment Strategies

For practitioners deploying LLM-based code generation tools, our findings provide concrete guidance:

Set Realistic Expectations: Organizations should expect 25–34% pass rates rather than synthetic benchmark success rates (84–89%) for real-world class-level generation. This means LLM-generated code requires mandatory human review and comprehensive testing rather than being production-ready.

Implement Adaptive RAG: The retrieval pipeline evaluated here (GraphCodeBERT embeddings, top-k=2 full implementations) provides clear benefits (4–7% improvement) specifically with partial documentation. Organizations

using a similar retrieval-augmented setup should consider enabling it when documentation is incomplete, disabling it when documentation is comprehensive, and validate that RAG-suggested imports and dependencies exist in the target context.

Focus Review on Dominant Error Types: Since `AttributeError` and `TypeError` account for two-thirds of failures, code review processes should prioritize checking object attribute access patterns and type consistency. Static analysis tools targeting these specific error types could efficiently catch the majority of LLM-generated mistakes.

8.3.4 Research Directions for Enhanced Code Generation

Our concentrated error patterns reveal specific research directions that could substantially improve generation capabilities:

Object-Oriented Semantic Learning: The dominance of `AttributeError` (43.84%) indicates that current training inadequately captures object-oriented concepts. Future research should investigate training objectives specifically targeting attribute resolution, inheritance relationships, and dynamic attribute handling. Graph neural network architectures that explicitly model class hierarchies may better capture object relationships than transformer architectures alone.

Type-Aware Generation Architectures: The persistent `TypeError` challenge (21.65%) motivates research into generation architectures that maintain explicit type information during generation, enforcing consistency requirements. Integration with gradual typing systems could provide training signals for type-correct generation.

Error-Specific Training Curricula: The concentrated three-error signature (`AttributeError`, `TypeError`, `AssertionError` accounting for 84% of failures) suggests targeted training curricula could efficiently improve generation quality. Curriculum learning approaches that progressively increase object-oriented complexity may better develop semantic understanding than uniform dataset training.

8.3.5 Methodological Contributions and Future Evaluation

This work establishes methodological advances for code generation research:

Real-World Evaluation Paradigm: *RealClassEval* demonstrates the feasibility and necessity of real-world code generation evaluation. The field requires systematic development of large-scale, diverse real-world benchmarks that preserve the complexity and authenticity missing from synthetic evaluations.

Error-Focused Analysis Framework: Moving beyond aggregate pass rates to detailed error type analysis reveals failure mechanisms rather than just failure rates. The RAG error substitution pattern—logic failures replaced by dependency errors—provides critical insights that would remain invisible in pass-rate-only evaluation.

Temporal Evaluation Strategy: The pre-cutoff/post-cutoff temporal split provides a rigorous approach to data contamination concerns while enabling systematic study of generalization patterns.

As LLM-based development tools become increasingly prevalent in software engineering workflows, rigorous evaluation on realistic tasks becomes essential. This work demonstrates that synthetic benchmarks provide false confidence about practical capabilities, while systematic real-world evaluation reveals both current limitations and specific paths forward. The class-level granularity investigated here provides the diagnostic foundation necessary for advancing both fundamental LLM capabilities and sophisticated repository-level approaches, ultimately contributing to more reliable and effective automated programming assistance.

9 Limitations and Future Work

While this study provides valuable insights into LLM class-level code generation, several limitations warrant discussion and motivate future research.

9.1 Limitations

9.1.1 Sample Size

The most significant limitation concerns dataset size and the resulting small to negligible effect sizes in the majority of cases. Our study evaluated 200 classes per *RealClassEval* version, totalling 400 real-world classes compared to ClassEval’s 100 synthetic tasks. While each split (pre-cutoff and post-cutoff) of *RealClassEval* is 2 times larger than the established ClassEval benchmark and represents the first carefully collected real-world class-level dataset, it remains relatively small by machine learning evaluation standards. This size limitation appears in consistently small effect sizes across research questions despite statistical significance. Docstring effects and RAG effects showed negligible practical differences, and error distribution comparisons between pre-cutoff and post-cutoff showed small effects compared to the synthetic-real divide. These small effect sizes reflect three factors: genuine small practical effects, high variability between classes, and limited statistical power to detect small-to-medium effects with 200 samples.

The limited size stems from practical constraints. Generating and testing over 16,000 classes across seven LLMs, two dataset versions, three docstring conditions, and two retrieval settings required substantial computational resources. At current API rates of different LLMs’ API service providers, the total generation cost, including trial and error, was approximately \$1000 for API calls alone, plus over 500 CPU hours for test execution. Each additional class requires LLM generation, test suite generation, and test execution, increasing costs directly with size. Expanding to 1,000+ classes per version would require proportional resource scaling, exceeding typical academic research budgets.

However, our sample size is sufficient for establishing fundamental patterns. The large performance gap between synthetic and real-world code and the error type differences are robust with clear practical significance. The small effect sizes for docstrings and RAG reflect genuine small benefits rather than merely insufficient power, as evidenced by consistency across multiple models, statistically significant findings in key comparisons, and coherent patterns. Critically, this study provides the first evidence from real-world data on class-level code generation, establishing baselines and patterns that synthetic benchmarks alone cannot reveal. Future work with larger-scale real-world datasets is essential for more conclusive findings, particularly for detecting smaller effects and enabling more detailed analyses.

9.1.2 Model Selection and Recency

Our evaluation included seven state-of-the-art models as of October 2025, representing diverse architectures and capabilities. However, the rapid pace of LLM development means newer models may show different patterns. The consistency of findings across all seven models—particularly the performance gap between synthetic and real-world code and error type dominance—suggests robust patterns likely to persist in future models. Additionally, all evaluated models are transformer-based architectures, and alternative architectures like graph neural networks explicitly modeling code structure may show different strengths, particularly for understanding object-oriented concepts.

9.1.3 Task Scope: Class-Level vs. Other Granularities

This study focuses exclusively on class-level code generation, where models must implement complete classes with multiple methods given only skeletons. Findings may not generalize to other granularities: single-function generation may show higher success rates due to a simpler context, while generating entire projects likely shows even larger challenges due to cross-file dependencies. The choice of class-level granularity reflects realistic software engineering tasks and enables controlled evaluation, but complete understanding requires evaluation across multiple granularities. The error pattern insights may not generalize to other object-oriented granularities (Tambon et al., 2025) and require validation.

9.1.4 Test Suite Generation Limitations

We relied on Pynguin for automatic test suite generation, using its DynaMOSA algorithm to achieve high branch coverage. While this approach enables objective correctness assessment without manual test writing, automatically generated test suites have limitations: they primarily assess that code executes without exceptions, but may not validate whether methods compute correct results; some classes may have low-coverage test suites missing errors in uncovered paths. This limitation is asymmetric across our two benchmark types:

PYNGUIN achieves 93.2% average branch coverage on ClassEval but only 58.7% on *RealClassEval*, meaning functional errors in uncovered branches are somewhat more likely to go undetected on *RealClassEval* than on ClassEval. Section 3.3.1 reports a manual test-quality assessment (n=78) confirming that generated test suites are overwhelmingly assertion-based (88.5%) rather than relying solely on implicit exception checks. This addresses whether tests contain explicit checks at all, but it does not establish that those assertions accurately capture the intended semantics of the original classes, nor does it measure the fault-detection capability (mutation adequacy) of the generated suites. Semantic and fault-detection adequacy of the automatically generated oracle therefore remains an open limitation of this study, one that manual test authoring or mutation-testing-based validation could help address in future work.

9.1.5 Prompt Engineering and Generation Settings

We used a standardized prompt across all models with consistent generation settings to ensure fair comparison, but this does not reflect optimal prompting strategies for each model. Some models may benefit from model-specific prompt formats, few-shot examples (Yuen et al., 2025), or interactive refinement strategies. However, the finding that all seven models show similar error patterns suggests that fundamental capabilities rather than prompt optimization drive performance. The goal was to assess base capabilities under standardized conditions rather than optimizing each model individually, reflecting realistic deployment scenarios.

9.1.6 Seen versus Unseen Benchmark Shelf Life

Our pre-cutoff (nominally seen) versus post-cutoff (nominally unseen) split is intended to approximate corpus-level exposure, but it rests on assumptions worth stating explicitly. CodeSearchNet, the source corpus underlying our pre-cutoff sampling, is a function-level dataset of (comment, code) pairs, not a corpus of complete classes; even where a model’s training data overlapped with CodeSearchNet, it would have encountered the constituent functions individually, not the class-level artifacts reconstructed for this study. Our seen versus unseen labelling therefore reflects exposure to related code patterns at a coarser granularity than the artifacts we evaluate, and should not be read as confirmed exposure to the specific classes under test. In the other direction, we cannot definitively verify that post-cutoff data is unseen: models may have encountered mirrors, forks, or near-duplicate code, and for closed models, stated knowledge cutoffs do not fully rule out post-training exposure, distillation from other models, or benchmark contamination through indirect channels. The small, largely non-significant differences we observe between pre-cutoff and post-cutoff performance are consistent with corpus-level familiarity not meaningfully improving real-world class-level generation, but given

the above, this should be treated as suggestive rather than a definitive test of memorization.

Separately, temporal-split benchmarks like *RealClassEval* have a limited shelf life by construction: repositories that are unseen at the time of dataset creation will eventually be scraped into future models’ training corpora as time passes. *RealClassEval*’s post-cutoff half should therefore be understood as unseen relative to the models evaluated in this study, not as a permanently unseen reference set. Reusing this benchmark to evaluate future models will require periodically refreshing the post-cutoff sample with newer repositories to preserve its intended function.

9.1.7 Generalization Beyond Python

This is a Python-only study, as suggested by the title of the paper. Therefore, all evaluated code is Python-based, drawn from open-source GitHub repositories. Python’s dynamic typing likely shapes our error-type findings; the dominance of `AttributeError` and `TypeError` among generation failures may reflect the absence of compile-time type checking rather than a language-agnostic property of class-level generation difficulty, and statically typed languages could exhibit a different error-type profile entirely. Closed-source or enterprise code may also follow conventions or domain-specific patterns under-represented in open-source training data, and the synthetic-versus-real-world performance gap we report may not transfer directly to those settings. This makes a multi-language and closed-source evaluation an important direction for future work.

9.1.8 Evaluation Without Repository Context

RealClassEval evaluates class implementation in isolation from the surrounding repository: dependencies, sibling classes, and project-level context available to a developer (or a repository-aware tool) are not provided to the model. This is a deliberate scope choice given that we aim to measure class-level implementation capability given a skeleton and docstrings, isolated from confounding effects of context availability. But this also means part of the synthetic-versus-real-world performance gap reported in this paper likely reflects the absence of contextual information rather than a pure deficit in object-oriented reasoning. Real-world deployment scenarios that supply repository context (e.g., via retrieval-augmented pipelines, as explored in RQ3, or repository-aware coding agents) may narrow this gap; our results should be interpreted as a lower bound on context-free class implementation capability rather than a ceiling on real-world task performance more broadly.

Despite these limitations, this study establishes fundamental patterns in LLM class-level code generation, provides the first real-world evaluation baseline, and illuminates specific research directions for advancing code synthesis capabilities. The limitations identified here directly motivate ongoing work

developing large-scale, diverse, quality-controlled real-world benchmarks to enable more conclusive future studies.

9.2 Future Work

The field urgently requires large-scale, diverse, real-world code generation benchmarks to enable definitive evaluation and advance beyond synthetic task limitations. An ideal benchmark would include:

- **Scale:** 1,000+ classes per temporal category (pre-cutoff, post-cutoff), enabling detection of small-to-medium effect sizes with high power (> 0.80 for Cohen’s $d = 0.2$) and supporting subgroup analyses by domain, complexity, or project characteristics.
- **Diversity:** Classes spanning multiple domains (web development, data science, systems programming, scientific computing), programming paradigms (object-oriented, functional, procedural mixtures), and complexity levels (simple data structures to complex business logic), ensuring generalization beyond specific niches.
- **Quality Control:** Systematic filtering for engineered projects with mature testing practices, automated test suite validation (ensuring tests actually execute and provide coverage), and manual spot-checking to verify task quality and remove problematic edge cases.
- **Metadata Richness:** Annotations for class complexity (cyclomatic complexity, coupling metrics), domain categories, required dependencies, and testing characteristics (unit vs. integration, coverage levels), enabling fine-grained analysis of where models succeed or fail.
- **Multi-Language Support:** Extension beyond Python to include Java, JavaScript, C++, and other languages, assessing whether findings generalize across language ecosystems or reflect Python-specific patterns.

Developing such a benchmark represents a substantial research contribution in itself, requiring substantial curation effort and computational costs. The insights from this study—particularly the importance of preserving realistic object hierarchies, dependencies, and testing practices—provide design principles for next-generation benchmarks.

10 Threats to Validity

We discuss threats to validity following established guidelines for empirical software engineering research, addressing internal validity (confounding factors), external validity (generalizability), and construct validity (measurement accuracy).

10.1 Internal Validity

Internal validity concerns whether observed effects genuinely result from our experimental design rather than confounding variables. We employed several controls to ensure valid findings. We used identical prompts across all models and consistent generation settings to control for prompt-induced variance. All test suites were generated using Pynguin with an identical configuration, ensuring consistent evaluation across classes. We used random sampling for class selection, reducing selection bias toward specific project types or domains. For RQ2 and RQ3, we employed within-subjects designs where each class was evaluated under all docstring conditions and both retrieval settings, controlling for snippet-specific difficulty.

10.2 External Validity

External validity concerns generalizability beyond our specific experimental setting. Several factors limit generalizability. All evaluated code is Python-based, and findings may not generalize to statically typed languages like Java or C++, which would likely show fewer type errors due to compile-time checking but potentially more complex type hierarchy challenges. All data derives from open-source GitHub repositories, potentially limiting generalizability to proprietary enterprise codebases, which may follow different conventions, have more comprehensive documentation, or employ domain-specific patterns unfamiliar to models trained predominantly on open-source data.

10.3 Construct Validity

Construct validity concerns whether measurements accurately capture intended constructs. We use pass rate as the primary correctness metric, which has limitations: it depends entirely on test suite quality, measures behavioural rather than structural correctness, and doesn't distinguish between semantically equivalent implementations. However, pass rate is standard in code generation research (Yeo et al., 2024; Hendrycks et al., 2021; Yuen et al., 2025), provides an objective functional correctness measurement through execution, and is validated across multiple test cases per class. Our qualitative analysis examined test suites and failure modes, confirming that pass rates meaningfully distinguish correct implementations from failures.

A related threat concerns the reliability of our reported coverage figures (93.2% *ClassEval*, 58.7% *RealClassEval*) as absolute measurements. A reproducibility check traced our *ClassEval* coverage shortfall relative to the 98.2% figure *ClassEval*'s own authors report (Du et al., 2023) to unresolved third-party dependencies in our measurement environment, which is a common artifact when re-executing real-world code at scale, and one likely present in the *RealClassEval* figure as well. We retain the originally reported values for consistency with our experimental logs and treat both as conservative estimates.

11 Conclusion

This work provides the first comprehensive evaluation of large language model performance on class-level code generation using real-world projects, revealing fundamental limitations masked by synthetic benchmark evaluation. Through systematic analysis classes generated by seven state-of-the-art models, we demonstrate that current LLMs achieve 84–89% correctness on synthetic benchmarks but only 25–34% on real-world code—a 53–62 percentage point gap. This performance drop stems from fundamentally different error patterns rather than just harder problems: synthetic benchmarks test logical correctness through simple checks, while real-world code exercises more complex object-oriented patterns, a difficulty not fully separable from the absence of repository-level context in our evaluation design. Real-world failures are dominated by **AttributeErrors** and **TypeErrors**, accounting for two-thirds of all failures, while synthetic benchmarks show mostly logic-induced errors. Modern LLMs have completely mastered Python syntax but struggle with object-oriented semantics. Models achieve nearly identical pass rates on seen and unseen real-world code, which we interpret as evidence against corpus-level familiarity being a major driver of real-world performance. As discussed in Section 9, however, this should not be read as a definitive test of memorization, given the function-level granularity of our seen-data corpus. This challenges current evaluation practices: synthetic benchmark performance does not accurately represent real-world capabilities, and organizations should expect much lower success rates when deploying LLM-based code generation tools.

Our studies reveal that documentation and retrieval provide complementary but modest improvements, with deployment of a similar retrieval setup likely requiring adaptive strategies based on documentation completeness. Beyond these immediate findings, this work addresses a critical gap in the code generation research landscape: understanding class-level capabilities as the essential foundation for repository-level approaches. The concentrated error patterns point to specific research directions: developing training objectives targeting attribute resolution and type consistency, creating context-aware retrieval systems that filter incompatible dependencies, and building large-scale real-world benchmarks spanning multiple languages and domains.

As LLM-based development tools become increasingly common in software engineering and repository-level approaches advance rapidly, a systematic understanding of class-level capabilities becomes essential. This work establishes that synthetic benchmarks provide false confidence about fundamental implementation abilities, while class-level evaluation on real-world code reveals both current limitations and specific paths forward. The diagnostic foundation provided here enables more effective development of repository-level approaches and more reliable deployment of automated programming assistance in production environments. We make our data and evaluation framework publicly available to facilitate continued research in this critical area.

Acknowledgments

We used AI-enabled tools (Claude and Grammarly) to assist with manuscript revisions. All AI-assisted content was reviewed and verified for accuracy by the first author.

Declarations

Funding

This research was supported by the **Natural Sciences and Engineering Research Council of Canada (NSERC)** under Grant Number: *CREATE 555406-2021*.

Conflict of Interest

Emad Shihab has ongoing collaborations with EMSE Editorial Board members: *Foutse Khomh*, *Sarah Nadi*, *Mei Nagappan*, and *Alexander Serebrenik*.

Ethical Approval

This article does not contain any studies with human participants or animals performed by any of the authors.

Informed Consent

Not applicable.

Author Contributions

Musfiqur Rahman: Conceptualization, Methodology, Implementation, Formal analysis, Investigation, Data curation, Writing - original draft, Writing - review & editing, Visualization. **SayedHassan Khatoonabadi:** Conceptualization, Methodology. **Emad Shihab:** Conceptualization, Writing - review, Supervision, Project administration, Funding acquisition.

Data Availability Statement

The datasets generated and analyzed during the current study, along with all code and analysis scripts, are publicly available in our replication package at: <https://github.com/mrsumitbd/RealClassEval-Replication>.

Clinical Trial Number

Not applicable.

References

- (2010) GitHub - tkaemming/django-subdomains: Subdomain helpers for the Django framework, including subdomain-based URL routing. — github.com. <https://github.com/tkaemming/django-subdomains>, [Accessed 20-03-2025]
- (2023) GitHub Copilot · Your AI pair programmer — github.com. <https://github.com/features/copilot>, [Accessed 01-11-2025]
- (2024) Codestral — Mistral AI — mistral.ai. <https://mistral.ai/news/codestral/>, [Accessed 28-10-2025]
- (2024) CodeWhisperer is becoming a part of Amazon Q Developer - CodeWhisperer — docs.aws.amazon.com. docs.aws.amazon.com/codewhisperer/latest/userguide/whisper-legacy.html, [Accessed 01-11-2025]
- (2024) Understand: The Software Developer's Multi-Tool — scitools.com. <https://scitools.com/>, [Version 7.0, Build 1217, Accessed 28-02-2025]
- (2025a) Introducing GPT-4.1 in the API — openai.com. <https://openai.com/index/gpt-4-1/>, [Accessed 28-10-2025]
- (2025b) Introducing GPT-5 - Resource — OpenAI Academy — academy.openai.com. <https://academy.openai.com/public/clubs/work-users-ynjqu/resources/intro-gpt-5>, [Accessed 28-10-2025]
- (2025) The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation — ai.meta.com. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, [Accessed 28-10-2025]
- Agarwal S, Ahmad L, Ai J, Altman S, Applebaum A, Arbus E, Arora RK, Bai Y, Baker B, Bao H, et al. (2025) gpt-oss-120b & gpt-oss-20b model card. arXiv preprint arXiv:250810925
- Austin J, Odena A, Nye M, Bosma M, Michalewski H, Dohan D, Jiang E, Cai C, Terry M, Le Q, et al. (2021) Program synthesis with large language models. arXiv preprint arXiv:210807732
- Benjamini Y, Hochberg Y (1995) Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal statistical society: series B (Methodological)* 57(1):289–300
- Boehm BW (1984) Software engineering economics. *IEEE transactions on Software Engineering* (1):4–21
- Cao J, Chen Z, Wu J, Cheung SC, Xu C (2024) Javabench: A benchmark of object-oriented code generation for evaluating large language models. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pp 870–882
- Carey V, Wang YG (2001) Mixed-effects models in s and s-plus

- Chen M, Tworek J, Jun H, Yuan Q, Pinto HPDO, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G, et al. (2021) Evaluating large language models trained on code. arXiv preprint arXiv:210703374
- Chen S, Wang R, Tian Y, Xie X, Song W, Huang R (2026) Repository-level solidity code generation with large language models: From prompting to fine-tuning. arXiv preprint arXiv:260619988
- Cliff N (1993) Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological bulletin* 114(3):494
- Cochran WG (1977) Sampling techniques, vol 3. wiley New York
- Cohen J (2013) Statistical power analysis for the behavioral sciences. routledge
- Conover WJ (1999) Practical nonparametric statistics. john wiley & sons
- Cramér H (1999) Mathematical methods of statistics, vol 9. Princeton university press
- Deshpande A, Agarwal A, Shet S, Iyer A, Kanade A, Bairi R, Parthasarathy S (2024) Class-level code generation from natural language using iterative, tool-enhanced reasoning over repository. arXiv preprint arXiv:240501573
- Du X, Liu M, Wang K, Wang H, Liu J, Chen Y, Feng J, Sha C, Peng X, Lou Y (2023) Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. arXiv preprint arXiv:230801861
- Fisher RA (1922) On the interpretation of χ^2 from contingency tables, and the calculation of p. *Journal of the royal statistical society* 85(1):87–94
- Fisher RA (1970) Statistical methods for research workers. In: Breakthroughs in statistics: Methodology and distribution, Springer, pp 66–70
- Friedman M (1937) The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the american statistical association* 32(200):675–701
- Friedman M (1940) A comparison of alternative tests of significance for the problem of m rankings. *The annals of mathematical statistics* 11(1):86–92
- Golman R, Loewenstein G (2018) Information gaps: A theory of preferences regarding the presence and absence of information. *Decision* 5(3):143
- Guo D, Ren S, Lu S, Feng Z, Tang D, Liu S, Zhou L, Duan N, Svyatkovskiy A, Fu S, et al. (2020) Graphcodebert: Pre-training code representations with data flow. arXiv preprint arXiv:200908366
- Guo D, Lu S, Duan N, Wang Y, Zhou M, Yin J (2022) Unixcoder: Unified cross-modal pre-training for code representation. arXiv preprint arXiv:220303850
- Hendrycks D, Basart S, Kadavath S, Mazeika M, Arora A, Guo E, Burns C, Puranik S, He H, Song D, et al. (2021) Measuring coding challenge competence with apps. arXiv preprint arXiv:210509938
- Hu C, Zeng W, Shi Y, Shen B, Gu X (2026) In line with context: Repository-level code generation via context inlining. arXiv preprint arXiv:260100376
- Hui B, Yang J, Cui Z, Yang J, Liu D, Zhang L, Liu T, Zhang J, Yu B, Lu K, et al. (2024) Qwen2. 5-coder technical report. arXiv preprint arXiv:240912186
- Husain H, Wu HH, Gazit T, Allamanis M, Brockschmidt M (2019) Codesearch-net challenge: Evaluating the state of semantic code search. arXiv preprint

- arXiv:190909436
- Jiang J, Wang F, Shen J, Kim S, Kim S (2024) A survey on large language models for code generation. arXiv preprint arXiv:240600515
- Jimenez CE, Yang J, Wettig A, Yao S, Pei K, Press O, Narasimhan K (2023) Swe-bench: Can language models resolve real-world github issues? arXiv preprint arXiv:231006770
- Kruskal WH, Wallis WA (1952) Use of ranks in one-criterion variance analysis. *Journal of the American statistical Association* 47(260):583–621
- Laird NM, Ware JH (1982) Random-effects models for longitudinal data. *Biometrics* pp 963–974
- Le Hai N, Nguyen DM, Bui ND (2024) Repoexec: Evaluate code generation with a repository-level executable benchmark. arXiv e-prints pp arXiv–2406
- Li J, Shi X, Zhang K, Li L, Li G, Tao Z, Liu F, Tao C, Jin Z (2025) Codetrag: Supportive code retrieval on bigraph for real-world code generation. arXiv preprint arXiv:250410046
- Li R, Allal LB, Zi Y, Muennighoff N, Kocetkov D, Mou C, Marone M, Akiki C, Li J, Chim J, et al. (2023) Starcoder: may the source be with you! arXiv preprint arXiv:230506161
- Liu A, Feng B, Xue B, Wang B, Wu B, Lu C, Zhao C, Deng C, Zhang C, Ruan C, et al. (2024) Deepseek-v3 technical report. arXiv preprint arXiv:241219437
- Liu J, Xia CS, Wang Y, Zhang L (2023) Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36:21558–21572
- Loewenstein G (1994) The psychology of curiosity: A review and reinterpretation. *Psychological bulletin* 116(1):75
- Lukasczyk S, Fraser G (2022) Pynguin: Automated unit test generation for python. In: *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pp 168–172
- Macbeth G, Razumiejczyk E, Ledesma RD (2011) Cliff’s delta calculator: A non-parametric effect size program for two groups of observations. *Universitas Psychologica* 10(2):545–555
- Mann HB, Whitney DR (1947) On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* pp 50–60
- Meng Q, Zhang X, Ren Z, Visser J (2025) When more retrieval hurts: Retrieval-augmented code review generation. arXiv preprint arXiv:251105302
- Munaiah N, Kroh S, Cabrey C, Nagappan M (2017) Curating github for engineered software projects. *Empirical Software Engineering* 22(6):3219–3253
- Orel D, Azizov D, Nakov P (2025) Codet-m4: Detecting machine-generated code in multi-lingual, multi-generator and multi-domain settings. arXiv preprint arXiv:250313733
- Panichella A, Kifetew FM, Tonella P (2017) Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. *IEEE Transactions on Software Engineering* 44(2):122–158

- Pearson K (1900) X. on the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science 50(302):157–175
- Rahman M, Khatoonabadi S, Abdellatif A, Shihab E (2024) Automatic detection of llm-generated code: A case study of claude 3 haiku. arXiv preprint arXiv:240901382
- Ronanki K, Arvidsson S, Axell J (2025) Prompt engineering guidelines for using large language models in requirements engineering. In: Euromicro Conference on Software Engineering and Advanced Applications, Springer, pp 245–262
- Shapiro SS, Wilk MB (1965) An analysis of variance test for normality (complete samples). *Biometrika* 52(3-4):591–611
- Shi E, Wang Y, Tao W, Du L, Zhang H, Han S, Zhang D, Sun H (2022) Race: Retrieval-augmented commit message generation. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing
- Shi E, Wang Y, Gu W, Du L, Zhang H, Han S, Zhang D, Sun H (2023) Cosoda: Effective contrastive learning for code search. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, pp 2198–2210
- Silverman BW (2018) Density estimation for statistics and data analysis. Routledge
- Sommerville I (2011) Software engineering (ed.). America: Pearson Education Inc
- Taherkhani H, Sepindband M, Pham HV, Wang S, Hemmati H (2024) Epic: Cost-effective search-based prompt engineering of llms for code generation. arXiv preprint arXiv:240811198
- Tambon F, Moradi-Dakhel A, Nikanjam A, Khomh F, Desmarais MC, Antoniol G (2025) Bugs in large language models generated code: An empirical study. *Empirical Software Engineering* 30(3):65
- Tao Y, Qin Y, Liu Y (2025) Retrieval-augmented code generation: A survey with focus on repository-level approaches. arXiv preprint arXiv:251004905
- Wang P, Zhang L, Liu F, Tao C, Zhu Y (2026) Codemem: Ast-guided adaptive memory for repository-level iterative code generation. arXiv preprint arXiv:260102868
- Wang Y, Le H, Gotmare AD, Bui ND, Li J, Hoi SC (2023) Codet5+: Open code large language models for code understanding and generation. arXiv preprint arXiv:230507922
- Wang Z, Zhou Z, Song D, Huang Y, Chen S, Ma L, Zhang T (2025) Towards understanding the characteristics of code generation errors made by large language models. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), IEEE Computer Society, pp 717–717
- Wang ZZ, Asai A, Yu XV, Xu FF, Xie Y, Neubig G, Fried D (2024) Coderag-bench: Can retrieval augment code generation? arXiv preprint arXiv:240614497

- Wilcoxon F (1992) Individual comparisons by ranking methods. In: Breakthroughs in statistics: Methodology and distribution, Springer, pp 196–202
- Wu T, Chen R, Du W, Ma S, Qi G, Xing Z, Khadivi S, Periyathambi R, Haffari G (2026) Environment-aware code generation: How far are we? arXiv preprint arXiv:260112262
- Xiao T, Fan Y, Calefato F, Treude C, Kula RG, Hata H, Baltes S (2025) Self-admitted genai usage in open-source software. arXiv preprint arXiv:250710422
- Yang G, Zhou Y, Cheng W, Zhang X, Chen X, Zhuo TY, Liu K, Zhou X, Lo D, Chen T (2024) Less is more: Docstring compression in code generation. arXiv preprint arXiv:241022793
- Yang Z, Chen S, Gao C, Li Z, Hu X, Liu K, Xia X (2025) An empirical study of retrieval-augmented code generation: Challenges and opportunities. ACM Transactions on Software Engineering and Methodology
- Yeo S, Ma YS, Kim SC, Jun H, Kim T (2024) Framework for evaluating code generation ability of large language models. Etri Journal 46(1):106–117
- Yu H, Shen B, Ran D, Zhang J, Zhang Q, Ma Y, Liang G, Li Y, Wang Q, Xie T (2024) Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp 1–12
- Yuen A, Pangas J, Polash MMH, Abdellatif A (2025) Prompting matters: Assessing the effect of prompting techniques on llm-generated class code. In: 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 803–808
- Zhang K, Li J, Li G, Shi X, Jin Z (2024) Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. arXiv preprint arXiv:240107339
- Zheng D, Wang Y, Shi E, Zhang R, Ma Y, Zhang H, Zheng Z (2024) Humanevo: An evolution-aware benchmark for more realistic evaluation of repository-level code generation. arXiv preprint arXiv:240606918

A Appendix

A.1 Repository filtering funnel

Table 13: Repository filtering funnel for Pre-Cutoff and Post-Cutoff datasets.

Filtering Criterion	Pre-Cutoff	Post-Cutoff
Initial Pool	13,590	91,048
Filtered on Project Size and Activity	5,348	13,197
Filtered on License Declaration	4,021	4,292
Filtered on Code Quality Indicators	2,970	1,735

A.2 Full Per-Model Statistical Tables

The following table provides the full per-model breakdown underlying the condensed summary Tables 2 and 3 in Section 4.

Table 14: Per-Class Pass Rate Analysis (full per-model breakdown). **Bold** entries represent statistical significance.

Model	Comparison	Mean Diff (95% CI)	FDR-Adjusted p for Mann-Whitney U Tests	Effect Size
Codestral	ClassEval vs. Pre-Cutoff	46.93 [39.46, 54.93]	1.0233e-10	Medium
	ClassEval vs. Post-Cutoff	51.50 [44.36, 58.63]	1.4623e-16	Large
	Pre-Cutoff vs. Post-Cutoff	4.57 [-4.32, 13.74]	0.4623	Negligible
Deepseek-V3	ClassEval vs. Pre-Cutoff	50.50 [42.82, 57.75]	1.9786e-11	Medium
	ClassEval vs. Post-Cutoff	49.38 [42.82, 56.03]	3.5676e-14	Large
	Pre-Cutoff vs. Post-Cutoff	-1.12 [-9.49, 7.59]	0.6604	Negligible
GPT-4.1	ClassEval vs. Pre-Cutoff	59.45 [52.51, 65.89]	6.5615e-19	Large
	ClassEval vs. Post-Cutoff	53.17 [47.17, 59.09]	1.7335e-19	Large
	Pre-Cutoff vs. Post-Cutoff	-6.28 [-13.98, 2.07]	0.0613	Negligible
GPT-5	ClassEval vs. Pre-Cutoff	61.24 [54.63, 68.16]	1.4079e-20	Large
	ClassEval vs. Post-Cutoff	57.05 [49.81, 63.58]	4.7559e-18	Large
	Pre-Cutoff vs. Post-Cutoff	-4.18 [-12.33, 3.59]	0.3013	Negligible
GPT-OSS	ClassEval vs. Pre-Cutoff	58.71 [51.48, 65.82]	4.7559e-18	Large
	ClassEval vs. Post-Cutoff	49.66 [42.19, 56.93]	7.1164e-16	Large
	Pre-Cutoff vs. Post-Cutoff	-9.05 [-17.54, -0.46]	0.0188	Negligible
Llama 4 Maverick	ClassEval vs. Pre-Cutoff	54.25 [46.70, 61.57]	4.4041e-16	Large
	ClassEval vs. Post-Cutoff	49.68 [42.60, 56.61]	7.1164e-16	Large
	Pre-Cutoff vs. Post-Cutoff	-4.57 [-12.92, 3.78]	0.1355	Negligible
Qwen Coder 2.5	ClassEval vs. Pre-Cutoff	50.02 [42.22, 57.07]	1.9965e-13	Large
	ClassEval vs. Post-Cutoff	43.96 [36.70, 51.25]	7.5609e-12	Medium
	Pre-Cutoff vs. Post-Cutoff	-6.07 [-14.40, 2.82]	0.1200	Negligible

Table 15: Pooled Chi-Square Analysis (full per-model breakdown).

Model	Comparison	Mean Diff. (95% CI)	FDR-Adjusted p	Effect Size
Codestral	ClassEval vs. Pre-Cutoff	51.75 [47.28, 56.83]	1.9032e-106	Medium
	ClassEval vs. Post-Cutoff	57.37 [53.22, 61.24]	1.7756e-168	Large
	Pre-Cutoff vs. Post-Cutoff	5.62 [-0.32, 11.33]	0.1034	Negligible
Deepseek-V3	ClassEval vs. Pre-Cutoff	54.44 [49.81, 59.06]	4.7305e-126	Medium
	ClassEval vs. Post-Cutoff	55.99 [52.14, 59.88]	7.6134e-174	Large
	Pre-Cutoff vs. Post-Cutoff	1.55 [-3.73, 7.40]	0.7654	Negligible
GPT-4.1	ClassEval vs. Pre-Cutoff	58.14 [52.86, 62.83]	2.3701e-152	Large
	ClassEval vs. Post-Cutoff	57.43 [53.83, 61.24]	1.8167e-193	Large
	Pre-Cutoff vs. Post-Cutoff	-0.71 [-6.43, 5.09]	0.8587	Negligible
GPT-5	ClassEval vs. Pre-Cutoff	60.36 [55.90, 65.02]	7.4220e-155	Large
	ClassEval vs. Post-Cutoff	58.06 [53.93, 61.48]	1.0034e-191	Large
	Pre-Cutoff vs. Post-Cutoff	-2.29 [-7.94, 3.10]	0.5707	Negligible
GPT-OSS	ClassEval vs. Pre-Cutoff	62.17 [57.34, 66.66]	2.2409e-149	Large
	ClassEval vs. Post-Cutoff	58.28 [54.47, 62.06]	7.6322e-179	Large
	Pre-Cutoff vs. Post-Cutoff	-3.89 [-9.75, 1.57]	0.2758	Negligible
Llama 4 Maverick	ClassEval vs. Pre-Cutoff	56.38 [51.37, 61.21]	3.4048e-123	Medium
	ClassEval vs. Post-Cutoff	57.25 [53.33, 60.92]	2.3489e-174	Large
	Pre-Cutoff vs. Post-Cutoff	0.87 [-4.96, 6.84]	0.8587	Negligible
Qwen Coder 2.5	ClassEval vs. Pre-Cutoff	52.85 [47.58, 57.67]	7.2034e-106	Medium
	ClassEval vs. Post-Cutoff	54.20 [50.31, 58.12]	1.2561e-152	Large
	Pre-Cutoff vs. Post-Cutoff	1.35 [-4.57, 7.58]	0.7779	Negligible